```
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/MainActivity.kt =====
package com.lightningstudio.watchrss.phone

import android.content.Intent
import android.os.Bundle
import androidx.activity.ComponentActivity

/**
 * 应用入口 Activity。
 * 接收所有外部 Intent（分享链接、打开文件等），并立即转发给 HomeActivity。
 * HomeActivity 作为首页根 Tab，根页面返回交给系统处理，以保留预测返回到桌面的预览动画。
 */
class MainActivity : ComponentActivity\(\) {

    override fun onCreate\(savedInstanceState: Bundle?\) {
        super.onCreate\(savedInstanceState\)
        startActivity\(HomeActivity.createIntent\(this, intent\)\)
        finish\(\)
        overridePendingTransition\(0, 0\)
    }

    override fun onNewIntent\(intent: Intent\) {
        super.onNewIntent\(intent\)
        startActivity\(HomeActivity.createIntent\(this, intent\)\)
        finish\(\)
        overridePendingTransition\(0, 0\)
    }
}
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/HomeActivity.kt =====
package com.lightningstudio.watchrss.phone

import android.content.Context
import android.content.Intent
import android.net.Uri
import android.os.Build
import android.os.Bundle
import android.os.SystemClock
import android.provider.OpenableColumns
import android.util.Log
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.result.contract.ActivityResultContracts
import androidx.activity.viewModels
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import android.widget.Toast
import androidx.core.content.ContextCompat
import androidx.lifecycle.lifecycleScope
import com.lightningstudio.watchrss.phone.data.backup.WATCHRSS_BACKUP_EXTENSION
import com.lightningstudio.watchrss.phone.data.backup.WATCHRSS_BACKUP_MIME_TYPE
import com.lightningstudio.watchrss.phone.ui.MainScreen
import com.lightningstudio.watchrss.phone.ui.theme.WatchRssPhoneTheme
import com.lightningstudio.watchrss.phone.platform.PlatformLinkRouter
import com.lightningstudio.watchrss.phone.viewmodel.MainViewModel
```

```
import com.lightningstudio.watchrss.phone.viewmodel.MainViewModelFactory
import com.lightningstudio.watchrss.phone.viewmodel.SharedImportPromptUi
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale

class HomeActivity : ComponentActivity\(\) {
    companion object {
        private const val TAG = "腕上RSS_Home"
        private val URL_PATTERN = Regex\("""https?://\\S+"""\)

        fun createIntent\(context: Context, inboundIntent: Intent? = null\): Intent {
            return Intent\(context, HomeActivity::class.java\).apply {
                inboundIntent?.let { putExtras\(it\) }
            }
        }
    }

    private val viewModel: MainViewModel by viewModels {
        val container = \(application as PhoneCompanionApplication\).container
        MainViewModelFactory\(
            container.repository,
            container.bluetoothSyncManager,
            container.usageTelemetry,
            container.backupService
        \)
    }

    private var pendingBluetoothAction: \(\(\) -> Unit\)? = null
    private var homeScreenStartedAt: Long = 0L

    private val bluetoothPermissionsLauncher =
        registerForActivityResult\(ActivityResultContracts.RequestMultiplePermissions\(\)\) { result ->
            if \(result.values.all { it }\) {
                pendingBluetoothAction?.invoke\(\)
            }
            pendingBluetoothAction = null
        }

    private val exportBluetoothLogLauncher =
        registerForActivityResult\(ActivityResultContracts.CreateDocument\("text/plain"\)\) { uri ->
            if \(uri == null\) {
                viewModel.showSyncStatusMessage\("已取消导出蓝牙日志"\)
                return@registerForActivityResult
            }
            lifecycleScope.launch {
                runCatching {
                    \(application as PhoneCompanionApplication\)
                        .container
                        .bluetoothDebugLog
                        .exportTo\(contentResolver, uri\)
                }.onSuccess { bytes ->
```

```
            viewModel.showSyncStatusMessage\("蓝牙日志已导出：$bytes 字节"\)
        }.onFailure { throwable ->
            Log.e\(TAG, "Failed to export bluetooth log", throwable\)
            viewModel.showSyncStatusError\("蓝牙日志导出失败：${throwable.message ?: "未知错误"}"\)
        }
    }
  }

private val exportBackupLauncher =
  registerForActivityResult\(ActivityResultContracts.CreateDocument\(WATCHRSS_BACKUP_MIME_TYPE\)\) { uri ->
    if \(uri == null\) {
      viewModel.showMessage\("已取消导出资料库"\)
      return@registerForActivityResult
    }
    viewModel.exportBackup\(uri.toString\(\)\)
  }

private val importLocalContentLauncher =
  registerForActivityResult\(ActivityResultContracts.OpenDocument\(\)\) { uri ->
    if \(uri == null\) {
      viewModel.showMessage\("已取消导入文件"\)
      return@registerForActivityResult
    }
    lifecycleScope.launch {
      runCatching {
        val fileName = withContext\(Dispatchers.IO\) {
          queryDisplayName\(uri\)
            ?: uri.lastPathSegment?.substringAfterLast\('/'\)
            ?: "未命名文件"
        }
        val mimeType = contentResolver.getType\(uri\)
        if \(isWrssBackup\(fileName, mimeType\)\) {
          runCatching {
            contentResolver.takePersistableUriPermission\(
              uri,
              Intent.FLAG_GRANT_READ_URI_PERMISSION
            \)
          }
          viewModel.inspectBackup\(fileName, uri.toString\(\)\)
        } else {
          val file = readSelectedLocalContent\(uri\)
          viewModel.importLocalContent\(
            fileName = file.fileName,
            mimeType = file.mimeType,
            bytes = file.bytes
          \)
        }
      }.onFailure { throwable ->
        Log.e\(TAG, "Failed to read local content", throwable\)
        viewModel.showError\("文件导入失败：${throwable.message ?: "未知错误"}"\)
      }
    }
  }

override fun onCreate\(savedInstanceState: Bundle?\) {
```

```
super.onCreate\(savedInstanceState\)
val container = \(application as PhoneCompanionApplication\).container

Log.i\(TAG, "=== 腕上RSS 手机端 已启动 ==="\)
container.bluetoothDebugLog.append\("MainActivity started"\)
Log.i\(TAG, "Package: $packageName"\)
runCatching { packageManager.getPackageInfo\(packageName, 0\) }
   .onSuccess { packageInfo ->
      Log.i\(TAG, "Version Code: ${packageInfo.longVersionCode}"\)
      Log.i\(TAG, "Version Name: ${packageInfo.versionName}"\)
   }
   .onFailure { throwable ->
      Log.w\(TAG, "Failed to resolve version info: ${throwable.message}"\)
   }
Log.i\(TAG, "==================================="\)

setContent {
   WatchRssPhoneTheme {
      val state by viewModel.uiState.collectAsState\(\)

      LaunchedEffect\(Unit\) {
         viewModel.toastEvent.collect { msg ->
            Toast.makeText\(this@HomeActivity, msg, Toast.LENGTH_SHORT\).show\(\)
         }
      }

      MainScreen\(
         uiState = state,
         showAccountFeatures = BuildConfig.DEBUG,
         onUrlChange = viewModel::updateUrlInput,
         onImportArticle = viewModel::importIndependentArticle,
         onAddRssSource = viewModel::addRssSource,
         onImportSharedLinkAsArticle = viewModel::importSharedLinkAsIndependent,
         onImportSharedLinkAsRss = viewModel::importSharedLinkAsRss,
         onConfirmSharedFileImport = ::importSharedFile,
         onDismissSharedImport = viewModel::dismissSharedImportPrompt,
         onSyncLibrary = { ensureBluetoothPermissions\(viewModel::syncLibraryByBluetooth\) },
         onSyncAccount = { ensureBluetoothPermissions\(viewModel::syncAccountByBluetooth\) },
         onOpenAccount = {
            startActivity\(AccountActivity.createIntent\(this@HomeActivity\)\)
         },
         onOpenSettings = {
            startActivity\(SettingsActivity.createIntent\(this@HomeActivity\)\)
         },
         onChooseBluetoothDevice = viewModel::chooseBluetoothDeviceForSync,
         onDismissBluetoothDevicePrompt = viewModel::dismissBluetoothDevicePrompt,
         onExportBluetoothLog = ::exportBluetoothLog,
         onOpenArticle = { article ->
            val platform = PlatformLinkRouter.detect\(article.url\)
            if \(platform != null\) {
               startActivity\(
                  PlatformWebViewActivity.createIntent\(
                     context = this@HomeActivity,
                     title = article.title.ifBlank { article.url },
                     url = article.url,
```

```
                        articleId = article.articleId,
                        initialReadingProgress = article.readingProgress
                    \)
                \)
            } else {
                startActivity\(ArticleReaderActivity.createIntent\(this@HomeActivity, article.articleId\)\)
            }
        },
        canOpenArticleInline = { article -> article.articleId.isNotBlank\(\) },
        onLoadArticleForInlineReader = container.repository::getArticle,
        onLoadImportedTextReaderForInlineReader = container.repository::getImportedTextReader,
        onLoadImportedTextChunkForInlineReader = container.repository::loadImportedTextChunk,
        onToggleFavorite = viewModel::toggleFavorite,
        onToggleWatchLater = viewModel::toggleWatchLater,
        onSaveArticleReadingProgress = { articleId, progress ->
            container.repository.updateArticleReadingProgress\(articleId, progress\)
        },
        onMoveRssSourceToTop = viewModel::moveRssSourceToTop,
        onReorderContentChannels = viewModel::reorderContentChannels,
        onToggleRssSourcePinned = viewModel::toggleRssSourcePinned,
        onDeleteRssSource = viewModel::deleteRssSource,
        onRefreshAllRssSources = viewModel::refreshAllRssSources,
        onRefreshRssSource = viewModel::refreshRssSource,
        onDeleteArticle = viewModel::deleteArticle,
        onExportBackup = ::exportBackup,
        onImportBackup = viewModel::importBackup,
        onRequestBackupReplace = viewModel::requestBackupReplaceConfirmation,
        onDismissBackupImport = viewModel::dismissBackupImportPrompt,
        onChooseConflictResolution = viewModel::chooseConflictResolution,
        onShowManualConflictOptions = viewModel::showManualConflictOptions,
        onDismissMessage = viewModel::clearMessage,
        onImportFile = ::selectLocalFile
    \)
  }
 }
 handleInboundIntent\(intent\)
}

override fun onResume\(\) {
 super.onResume\(\)
 homeScreenStartedAt = SystemClock.elapsedRealtime\(\)
 \(application as PhoneCcompanionApplication\).container.usageTelemetry.recordScreenOpen\("phone_home"\)
}

override fun onPause\(\) {
 super.onPause\(\)
 val startedAt = homeScreenStartedAt
 if \(startedAt > 0L\) {
  \(application as PhoneCcompanionApplication\).container.usageTelemetry.recordScreenDuration\(
   "phone_home",
   SystemClock.elapsedRealtime\(\) - startedAt
  \)
  homeScreenStartedAt = 0L
 }
}
```

```
override fun onNewIntent\(intent: Intent\) {
    super.onNewIntent\(intent\)
    setIntent\(intent\)
    handleInboundIntent\(intent\)
}

private fun ensureBluetoothPermissions\(action: \(\) -> Unit\) {
    val permissions = buildList {
        if \(android.os.Build.VERSION.SDK_INT >= android.os.Build.VERSION_CODES.S\) {
            add\(android.Manifest.permission.BLUETOOTH_CONNECT\)
        }
    }
    val missing = permissions.filter {
        ContextCompat.checkSelfPermission\(this, it\) != android.content.pm.PackageManager.PERMISSION_GRANTED
    }
    if \(missing.isEmpty\(\)\) {
        action\(\)
        return
    }
    pendingBluetoothAction = action
    bluetoothPermissionsLauncher.launch\(missing.toTypedArray\(\)\)
}

private fun exportBluetoothLog\(\) {
    val fileName = "watchrss-phone-bluetooth-${System.currentTimeMillis\(\)}.txt"
    exportBluetoothLogLauncher.launch\(fileName\)
}

private fun exportBackup\(\) {
    val timestamp = SimpleDateFormat\("yyyyMMdd-HHmmss", Locale.US\).format\(Date\(\)\)
    exportBackupLauncher.launch\("WatchRSS-backup-$timestamp$WATCHRSS_BACKUP_EXTENSION"\)
}

private fun selectLocalFile\(\) {
    importLocalContentLauncher.launch\(
        arrayOf\(
            "text/plain",
            "text/*",
            "application/epub+zip",
            WATCHRSS_BACKUP_MIME_TYPE,
            "application/zip",
            "application/octet-stream",
            "*/*"
        \)
    \)
}

private fun importSharedFile\(prompt: SharedImportPromptUi\) {
    val uri = runCatching { Uri.parse\(prompt.uriString\) }.getOrNull\(\)
    if \(uri == null\) {
        viewModel.dismissSharedImportPrompt\(\)
        viewModel.showError\("文件地址无效"\)
        return
    }
```

```
    viewModel.dismissSharedImportPrompt\(\)
    viewModel.showMessage\("正在读取文件…"\)
    lifecycleScope.launch {
        runCatching {
            readSelectedLocalContent\(uri, prompt.mimeType\)
        }.onSuccess { file ->
            viewModel.importLocalContent\(
                fileName = file.fileName,
                mimeType = file.mimeType,
                bytes = file.bytes
            \)
        }.onFailure { throwable ->
            Log.e\(TAG, "Failed to read shared local content", throwable\)
            viewModel.showError\("文件导入失败：${throwable.message ?: "未知错误"}"\)
        }
    }
}

private suspend fun readSelectedLocalContent\(
    uri: Uri,
    fallbackMimeType: String? = null
\): SelectedLocalContent =
    withContext\(Dispatchers.IO\) {
        val fileName = queryDisplayName\(uri\)
            ?: uri.lastPathSegment?.substringAfterLast\('/'\)
            ?: "未命名文件"
        val mimeType = contentResolver.getType\(uri\) ?: fallbackMimeType
        val bytes = contentResolver.openInputStream\(uri\)
            ?.use { input -> input.readBytes\(\) }
            ?: error\("无法读取文件"\)
        SelectedLocalContent\(fileName, mimeType, bytes\)
    }

private fun queryDisplayName\(uri: Uri\): String? {
    return runCatching {
        contentResolver.query\(uri, arrayOf\(OpenableColumns.DISPLAY_NAME\), null, null, null\)
            ?.use { cursor ->
                if \(!cursor.moveToFirst\(\)\) return@use null
                val index = cursor.getColumnIndex\(OpenableColumns.DISPLAY_NAME\)
                if \(index >= 0\) cursor.getString\(index\) else null
            }
    }.getOrNull\(\)?.takeIf { it.isNotBlank\(\) }
}

private fun handleInboundIntent\(intent: Intent?\) {
    if \(intent == null\) return
    val url = extractInboundUrl\(intent\)
    val fileUri = extractInboundFileUri\(intent\)
    if \(fileUri != null\) {
        lifecycleScope.launch {
            var inspectError: Throwable? = null
            val handled = runCatching {
                showInboundFilePrompt\(fileUri, intent.type\)
            }.onFailure { throwable ->
                inspectError = throwable
```

```
            Log.e\(TAG, "Failed to inspect shared local content", throwable\)
        }.getOrDefault\(false\)
        if \(!handled\) {
            if \(url != null\) {
                viewModel.showSharedLinkPrompt\(url\)
            } else {
                val errorMessage = inspectError?.message
                viewModel.showContentError\(
                    if \(errorMessage.isNullOrBlank\(\)\) {
                        "只支持导入 TXT 或 EPUB 文件"
                    } else {
                        "无法读取文件：$errorMessage"
                    }
                \)
            }
        }
    }
    return
  }
  if \(url != null\) {
    viewModel.showSharedLinkPrompt\(url\)
  }
}

private fun extractInboundUrl\(intent: Intent?\): String? {
  if \(intent == null\) return null
  val text = when \(intent.action\) {
    Intent.ACTION_VIEW -> intent.dataString?.takeIf { value ->
      value.startsWith\("http://"\) || value.startsWith\("https://"\)
    }
    Intent.ACTION_SEND -> intent.getCharSequenceExtra\(Intent.EXTRA_TEXT\)?.toString\(\)
      ?: firstClipText\(intent\)
    else -> null
  }
  return text?.lineSequence\(\)
    ?.firstOrNull { line ->
      line.contains\("http://"\) || line.contains\("https://"\)
    }
    ?.let { line ->
      URL_PATTERN.find\(line\)?.value?.trimUrlTail\(\) ?: line.trim\(\)
    }
}

private fun extractInboundFileUri\(intent: Intent\): Uri? {
  return when \(intent.action\) {
    Intent.ACTION_VIEW -> intent.data?.takeIf\(::isReadableLocalUri\)
    Intent.ACTION_SEND -> streamUriExtra\(intent\)?.takeIf\(::isReadableLocalUri\)
      ?: firstClipUri\(intent\)?.takeIf\(::isReadableLocalUri\)
    else -> null
  }
}

private suspend fun showInboundFilePrompt\(uri: Uri, intentMimeType: String?\): Boolean {
  val file = withContext\(Dispatchers.IO\) {
    val fileName = queryDisplayName\(uri\)
```

```
            ?: uri.lastPathSegment?.substringAfterLast('/')
            ?: "未命名文件"
        val mimeType = contentResolver.getType(uri) ?: intentMimeType
        if (!isSupportedLocalContent(fileName, mimeType)) return@withContext null
        InboundLocalFile(
            fileName = fileName,
            mimeType = mimeType,
            uriString = uri.toString()
        )
    } ?: return false
    viewModel.showSharedFilePrompt(
        fileName = file.fileName,
        mimeType = file.mimeType,
        uriString = file.uriString
    )
    return true
}

private fun streamUriExtra(intent: Intent): Uri? {
    return if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        intent.getParcelableExtra(Intent.EXTRA_STREAM, Uri::class.java)
    } else {
        @Suppress("DEPRECATION")
        intent.getParcelableExtra(Intent.EXTRA_STREAM) as? Uri
    }
}

private fun firstClipUri(intent: Intent): Uri? {
    val clipData = intent.clipData ?: return null
    return (0 until clipData.itemCount)
        .asSequence()
        .mapNotNull { index -> clipData.getItemAt(index).uri }
        .firstOrNull()
}

private fun firstClipText(intent: Intent): String? {
    val clipData = intent.clipData ?: return null
    return (0 until clipData.itemCount)
        .asSequence()
        .mapNotNull { index -> clipData.getItemAt(index).text?.toString() }
        .firstOrNull { it.isNotBlank() }
}

private fun isReadableLocalUri(uri: Uri): Boolean {
    val scheme = uri.scheme?.lowercase() ?: return false
    return scheme == "content" || scheme == "file"
}

private fun isWrssBackup(fileName: String, mimeType: String?): Boolean {
    return fileName.endsWith(WATCHRSS_BACKUP_EXTENSION, ignoreCase = true) ||
        mimeType.equals(WATCHRSS_BACKUP_MIME_TYPE, ignoreCase = true)
}

private fun isSupportedLocalContent(fileName: String, mimeType: String?): Boolean {
    val lowerName = fileName.lowercase()
```

```
        val lowerMime = mimeType.orEmpty\(\).lowercase\(\)
        return lowerName.endsWith\(".txt"\) ||
            lowerName.endsWith\(".epub"\) ||
            lowerMime.startsWith\("text/"\) ||
            lowerMime == "application/epub+zip"
    }

    private fun String.trimUrlTail\(\): String =
        trimEnd\('.', ',', ';', ':', '!', '?', '\)', ']', '}', '>', '。', '，', '、', '；', '：', '！', '？', '）', '】', '》' '\)
}

private data class InboundLocalFile\(
    val fileName: String,
    val mimeType: String?,
    val uriString: String
\)
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/ui/MainScreen.kt =====
package com.lightningstudio.watchrss.phone.ui

import androidx.activity.compose.PredictiveBackHandler
import androidx.compose.animation.AnimatedContent
import androidx.compose.animation.AnimatedVisibility
import androidx.compose.animation.fadeIn
import androidx.compose.animation.fadeOut
import androidx.compose.animation.slideInHorizontally
import androidx.compose.animation.slideOutHorizontally
import androidx.compose.animation.togetherWith
import androidx.compose.animation.core.Animatable
import androidx.compose.animation.core.FastOutSlowInEasing
import androidx.compose.animation.core.animate
import androidx.compose.animation.core.tween
import androidx.compose.foundation.ExperimentalFoundationApi
import androidx.compose.foundation.Image
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.RowScope
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxHeight
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.statusBarsPadding
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.LazyListState
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.lazy.rememberLazyListState
import androidx.compose.foundation.pager.HorizontalPager
import androidx.compose.foundation.pager.rememberPagerState
```

```
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material.icons.automirrored.filled.Article
import androidx.compose.material.icons.automirrored.filled.MenuBook
import androidx.compose.material.icons.automirrored.filled.OpenInNew
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.AccountCircle
import androidx.compose.material.icons.filled.Bookmark
import androidx.compose.material.icons.filled.BookmarkBorder
import androidx.compose.material.icons.filled.BugReport
import androidx.compose.material.icons.filled.Archive
import androidx.compose.material.icons.filled.Close
import androidx.compose.material.icons.filled.Delete
import androidx.compose.material.icons.filled.Description
import androidx.compose.material.icons.filled.Favorite
import androidx.compose.material.icons.filled.FavoriteBorder
import androidx.compose.material.icons.filled.FileOpen
import androidx.compose.material.icons.filled.Home
import androidx.compose.material.icons.filled.MoreVert
import androidx.compose.material.icons.filled.PushPin
import androidx.compose.material.icons.filled.RssFeed
import androidx.compose.material.icons.filled.Settings
import androidx.compose.material.icons.filled.Sync
import androidx.compose.material.icons.filled.VerticalAlignTop
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.CenterAlignedTopAppBar
import androidx.compose.material3.DropdownMenu
import androidx.compose.material3.DropdownMenuItem
import androidx.compose.material3.ElevatedCard
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.ExtendedFloatingActionButton
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.LinearProgressIndicator
import androidx.compose.material3.ListItem
import androidx.compose.material3.ListItemDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.NavigationBar
import androidx.compose.material3.NavigationBarItem
import androidx.compose.material3.NavigationRail
import androidx.compose.material3.NavigationRailItem
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.material3.TopAppBarDefaults
import androidx.compose.material3.VerticalDivider
import androidx.compose.material3.pulltorefresh.PullToRefreshDefaults
import androidx.compose.material3.pulltorefresh.PullToRefreshBox
import androidx.compose.material3.pulltorefresh.rememberPullToRefreshState
```

```
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableFloatStateOf
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.movableContentOf
import androidx.compose.runtime.produceState
import androidx.compose.runtime.remember
import androidx.compose.runtime.rememberCoroutineScope
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.runtime.setValue
import androidx.compose.runtime.SideEffect
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clipToBounds
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.asImageBitmap
import androidx.compose.ui.graphics.graphicsLayer
import androidx.compose.ui.layout.Layout
import androidx.compose.ui.platform.LocalUriHandler
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.compose.ui.zIndex
import com.lightningstudio.watchrss.phone.ArticleContentNodesKey
import com.lightningstudio.watchrss.phone.ArticleContentNodesSnapshot
import com.lightningstudio.watchrss.phone.ArticleReaderScreen
import com.lightningstudio.watchrss.phone.PlatformWebViewScreen
import com.lightningstudio.watchrss.phone.generateQRCode
import com.lightningstudio.watchrss.phone.connection.bluetooth.PhoneSyncConflictResolution
import com.lightningstudio.watchrss.phone.data.backup.BackupImportMode
import com.lightningstudio.watchrss.phone.data.db.PhoneArticleEntity
import com.lightningstudio.watchrss.phone.data.db.PhoneRssSourceEntity
import com.lightningstudio.watchrss.phone.data.model.ImportedContentIds
import com.lightningstudio.watchrss.phone.data.repo.PhoneImportedTextReader
import com.lightningstudio.watchrss.phone.platform.OnlineNovelLinkDetector
import com.lightningstudio.watchrss.phone.platform.PlatformLinkRouter
import com.lightningstudio.watchrss.phone.viewmodel.MainBluetoothDevicePromptUi
import com.lightningstudio.watchrss.phone.viewmodel.MainBluetoothDeviceUi
import com.lightningstudio.watchrss.phone.viewmodel.BackupImportPromptUi
import com.lightningstudio.watchrss.phone.viewmodel.MainConflictPromptUi
import com.lightningstudio.watchrss.phone.viewmodel.MainSyncProgressUi
import com.lightningstudio.watchrss.phone.viewmodel.MainUiState
import com.lightningstudio.watchrss.phone.viewmodel.SharedImportPromptKind
import com.lightningstudio.watchrss.phone.viewmodel.SharedImportPromptUi
import com.lightningstudio.watchrss.phone.ui.theme.roundedClickable
import com.lightningstudio.watchrss.phone.ui.theme.roundedCombinedClickable
import kotlinx.coroutines.CancellationException
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.collect
import kotlinx.coroutines.launch
import java.text.DateFormat
import java.util.Date
import kotlin.math.roundToInt
```

```
import sh.calvin.reorderable.ReorderableItem
import sh.calvin.reorderable.rememberReorderableLazyListState

private enum class MainPage {
    DASHBOARD,
    RSS,
    IMPORTS,
    CHANNEL
}

private val TopLevelMainPages = listOf\(
    MainPage.DASHBOARD,
    MainPage.RSS,
    MainPage.IMPORTS
\)

private fun MainPage.topLevelIndex\(\): Int = TopLevelMainPages.indexOf\(this\).coerceAtLeast\(0\)

private enum class UrlDialogMode {
    ARTICLE,
    RSS
}

private const val CONTENT_SOURCE_PREFIX = "source:"
private const val CONTENT_CHANNEL_FAVORITES = "virtual:favorites"
private const val CONTENT_CHANNEL_WATCH_LATER = "virtual:watch_later"
private const val CONTENT_CHANNEL_INDEPENDENT = "virtual:independent"
private const val CONTENT_CHANNEL_IMPORTED_TEXT = "virtual:imported_text"
private const val CHANNEL_TRANSITION_MS = 260
private const val CHANNEL_PREDICTIVE_EXIT_MS = 480
private const val CHANNEL_PREDICTIVE_EXIT_PROGRESS = 2f
private const val TAB_PREDICTIVE_EXIT_MS = 480
private const val TAB_PREDICTIVE_EXIT_PROGRESS = 1f
private const val READER_LEFT_PANE_RETURN_TRANSITION_MS = 480
private const val READER_FULLSCREEN_BACK_SETTLE_MS = 480
private const val ARTICLE_CARD_TITLE_LINES = 2
private const val ONLINE_NOVEL_IMPORT_WARNING =
    "小说导入仅支持txt/epub等开放格式本地文件，不支持来自在线小说库的阅读链接分享（七猫/番茄/晋江/起点等平台均不支持），仍然要将页面作为网页或RSS
private const val WATCH_RSS_QQ_GROUP_URL = "https://qm.qq.com/q/cJNTQuxfoW"
private const val WATCH_RSS_QQ_GROUP_NUMBER = "1083518433"
private val MainNavigationRailWidth = 80.dp

@Composable
private fun defaultMainElevatedCardColors\(\) = CardDefaults.elevatedCardColors\(\)

@Composable
private fun pinnedMainContentChannelContainerColor\(\): Color {
    val isDark = isSystemInDarkTheme\(\)
    return lerpMainColor\(
        start = MaterialTheme.colorScheme.surface,
        stop = if \(isDark\) Color.White else Color.Black,
        progress = if \(isDark\) 0.06f else 0.04f
    \)
}
```

```
private data class ReaderLeftPaneReturnState\(
    val channelKey: String,
    val articleId: String,
    val returnPage: MainPage,
    val initialProgress: Float = 0f
\)

private data class MainContentChannel\(
    val key: String,
    val title: String,
    val supportingText: String,
    val articleCount: Int,
    val source: PhoneRssSourceEntity? = null,
    val reorderSourceUrl: String? = source?.url,
    val articles: List<PhoneArticleEntity> = emptyList\(\),
    val icon: MainContentChannelIcon,
    val emptyTitle: String,
    val emptyText: String,
    val canRefresh: Boolean = false,
    val canDrag: Boolean = false,
    val dragGroup: MainContentDragGroup = MainContentDragGroup.FIXED,
    val sortOrder: Long = 0L
\)

private data class MainContentReorderRequest\(
    val sourceUrlsInDisplayOrder: List<String>,
    val independentIndex: Int?
\)

private data class InlineReaderPaneInput\(
    val article: PhoneArticleEntity,
    val readerArticle: PhoneArticleEntity?,
    val importedTextReader: PhoneImportedTextReader?,
    val onLoadImportedTextChunk: suspend \(String, Int\) -> String?,
    val onSaveReadingProgress: suspend \(Float\) -> Unit,
    val onBack: \(\) -> Unit,
    val onOpenImportedArticle: \(String\) -> Unit,
    val onOpenOriginal: \(String\) -> Unit,
    val listState: LazyListState,
    val contentReady: Boolean,
    val contentNodesCache: MutableMap<ArticleContentNodesKey, ArticleContentNodesSnapshot>,
    val positionAlreadyRestored: Boolean,
    val onPositionRestored: \(String\) -> Unit,
    val fullscreen: Boolean,
    val showFullscreenControl: Boolean,
    val onToggleFullscreen: \(\) -> Unit
\)

private enum class MainContentChannelIcon {
    RSS,
    BOOK,
    FAVORITE,
    WATCH_LATER,
    ARTICLE,
    IMPORTED
```

```
}

private enum class MainContentDragGroup {
    FIXED,
    PINNED,
    NORMAL
}

private sealed interface RecentImportEntry {
    val key: String
    val sortAt: Long

    data class Channel\(
        val channel: MainContentChannel,
        override val sortAt: Long
    \) : RecentImportEntry {
        override val key: String = "channel:${channel.key}"
    }

    data class Article\(
        val article: PhoneArticleEntity
    \) : RecentImportEntry {
        override val key: String = "article:${article.articleId}"
        override val sortAt: Long = maxOf\(article.importedAt, article.updatedAt\)
    }
}

@OptIn\(ExperimentalMaterial3Api::class, ExperimentalFoundationApi::class\)
@Composable
fun MainScreen\(
    uiState: MainUiState,
    showAccountFeatures: Boolean = true,
    onUrlChange: \(String\) -> Unit,
    onImportArticle: \(\) -> Unit,
    onImportFile: \(\) -> Unit,
    onAddRssSource: \(\) -> Unit,
    onImportSharedLinkAsArticle: \(String\) -> Unit,
    onImportSharedLinkAsRss: \(String\) -> Unit,
    onConfirmSharedFileImport: \(SharedImportPromptUi\) -> Unit,
    onDismissSharedImport: \(\) -> Unit,
    onSyncLibrary: \(\) -> Unit,
    onSyncAccount: \(\) -> Unit,
    onOpenAccount: \(\) -> Unit,
    onOpenSettings: \(\) -> Unit,
    onChooseBluetoothDevice: \(MainBluetoothDeviceUi\) -> Unit,
    onDismissBluetoothDevicePrompt: \(\) -> Unit,
    onExportBluetoothLog: \(\) -> Unit,
    onOpenArticle: \(PhoneArticleEntity\) -> Unit,
    canOpenArticleInline: \(PhoneArticleEntity\) -> Boolean,
    onLoadArticleForInlineReader: suspend \(String\) -> PhoneArticleEntity?,
    onLoadImportedTextReaderForInlineReader: suspend \(String\) -> PhoneImportedTextReader?,
    onLoadImportedTextChunkForInlineReader: suspend \(String, Int\) -> String?,
    onToggleFavorite: \(PhoneArticleEntity\) -> Unit,
    onToggleWatchLater: \(PhoneArticleEntity\) -> Unit,
    onSaveArticleReadingProgress: suspend \(String, Float\) -> Unit,
```

```
    onMoveRssSourceToTop: \(PhoneRssSourceEntity\) -> Unit,
    onReorderContentChannels: \(List<String>, Int?\) -> Unit,
    onToggleRssSourcePinned: \(PhoneRssSourceEntity\) -> Unit,
    onDeleteRssSource: \(PhoneRssSourceEntity\) -> Unit,
    onRefreshAllRssSources: \(\) -> Unit,
    onRefreshRssSource: \(PhoneRssSourceEntity\) -> Unit,
    onDeleteArticle: \(PhoneArticleEntity\) -> Unit,
    onExportBackup: \(\) -> Unit,
    onImportBackup: \(BackupImportMode\) -> Unit,
    onRequestBackupReplace: \(\) -> Unit,
    onDismissBackupImport: \(\) -> Unit,
    onChooseConflictResolution: \(PhoneSyncConflictResolution\) -> Unit,
    onShowManualConflictOptions: \(\) -> Unit,
    onDismissMessage: \(\) -> Unit
\) {
    var selectedContentChannelKey by rememberSaveable { mutableStateOf<String?>\(null\) }
    var selectedReaderArticleId by rememberSaveable { mutableStateOf<String?>\(null\) }
    var readerFullscreenActive by rememberSaveable { mutableStateOf\(false\) }
    var readerBackProgress by remember { mutableFloatStateOf\(0f\) }
    var readerFullscreenBackProgress by remember { mutableFloatStateOf\(0f\) }
    var readerBackAnimating by remember { mutableStateOf\(false\) }
    var readerOpenProgress by remember { mutableFloatStateOf\(1f\) }
    var readerOpenAnimating by remember { mutableStateOf\(false\) }
    var inlineReaderRestoredArticleId by remember { mutableStateOf<String?>\(null\) }
    var readerLeftPaneReturnState by remember { mutableStateOf<ReaderLeftPaneReturnState?>\(null\) }
    var lastContentChannelKey by rememberSaveable { mutableStateOf<String?>\(null\) }
    var channelReturnPageName by rememberSaveable { mutableStateOf\(MainPage.RSS.name\) }
    var channelBackProgress by remember { mutableFloatStateOf\(0f\) }
    var tabBackProgress by remember { mutableFloatStateOf\(0f\) }
    var tabBackActive by remember { mutableStateOf\(false\) }
    var suppressNextTopLevelTransition by remember { mutableStateOf\(false\) }
    var channelTabSwitchDestinationName by remember { mutableStateOf<String?>\(null\) }
    var currentTopLevelPageName by rememberSaveable { mutableStateOf\(MainPage.DASHBOARD.name\) }
    var isMediumOrExpandedLayout by remember { mutableStateOf\(false\) }
    var topLevelNavigationTargetName by remember { mutableStateOf<String?>\(null\) }
    var channelTabSwitchProgress by remember { mutableFloatStateOf\(0f\) }
    var channelTabSwitchDirection by remember { mutableFloatStateOf\(1f\) }
    var suppressNextChannelExit by remember { mutableStateOf\(false\) }
    var urlDialogMode by remember { mutableStateOf<UrlDialogMode?>\(null\) }
    val pagerState = rememberPagerState\(initialPage = MainPage.DASHBOARD.topLevelIndex\(\)\) {
        TopLevelMainPages.size
    }
    val contentPageListState = rememberLazyListState\(\)
    val importsPageListState = rememberLazyListState\(\)
    val channelArticleListState = rememberSaveable\(
        selectedContentChannelKey,
        saver = LazyListState.Saver
    \) {
        LazyListState\(\)
    }
    val coroutineScope = rememberCoroutineScope\(\)
    val uriHandler = LocalUriHandler.current

    val articlesBySource = remember\(uiState.rssArticles\) {
        uiState.rssArticles.groupBy { it.rssSourceUrl.orEmpty\(\) }
```

```
}
val contentChannels = remember\(uiState, articlesBySource\) {
    buildContentChannels\(uiState, articlesBySource\)
}
val importedContentChannelKey = contentChannels.firstOrNull { channel ->
    channel.source?.url?.let\(ImportedContentIds::isImportedTextSourceUrl\) == true
}?.key ?: CONTENT_CHANNEL_IMPORTED_TEXT
val selectedContentChannel = selectedContentChannelKey?.let { key ->
    contentChannels.firstOrNull { it.key == key }
}
val selectedReaderArticleListItem = selectedReaderArticleId?.let { articleId ->
    selectedContentChannel?.articles?.firstOrNull { it.articleId == articleId }
        ?: uiState.independentArticles.firstOrNull { it.articleId == articleId }
        ?: uiState.importedContentArticles.firstOrNull { it.articleId == articleId }
        ?: uiState.favorites.firstOrNull { it.articleId == articleId }
        ?: uiState.watchLater.firstOrNull { it.articleId == articleId }
        ?: uiState.rssArticles.firstOrNull { it.articleId == articleId }
}
val inlineReaderOpenSettled = selectedReaderArticleId != null &&
    !readerOpenAnimating &&
    readerOpenProgress >= 1f
val inlineReaderContentReady = inlineReaderOpenSettled
val inlineReaderLoadArticleId = selectedReaderArticleId
val hydratedSelectedReaderArticle by produceState<PhoneArticleEntity?>\(
    initialValue = null,
    inlineReaderLoadArticleId
\) {
    val articleId = inlineReaderLoadArticleId
    value = null
    if \(articleId != null\) {
        value = runCatching { onLoadArticleForInlineReader\(articleId\) }.getOrNull\(\)
    }
}
val selectedReaderArticle = hydratedSelectedReaderArticle
    ?.takeIf { it.articleId == selectedReaderArticleId }
    ?: selectedReaderArticleListItem
val selectedImportedTextReader by produceState<PhoneImportedTextReader?>\(
    initialValue = null,
    inlineReaderLoadArticleId
\) {
    val articleId = inlineReaderLoadArticleId
    value = null
    if \(articleId != null\) {
        value = runCatching { onLoadImportedTextReaderForInlineReader\(articleId\) }.getOrNull\(\)
    }
}
LaunchedEffect\(selectedContentChannelKey\) {
    selectedContentChannelKey?.let { key ->
        lastContentChannelKey = key
        channelBackProgress = 0f
    }
}
LaunchedEffect\(selectedReaderArticleId, selectedReaderArticle\) {
    if \(selectedReaderArticleId != null && selectedReaderArticle == null\) {
        selectedReaderArticleId = null
```

```
        readerFullscreenActive = false
        readerFullscreenBackProgress = 0f
        readerOpenProgress = 1f
        readerOpenAnimating = false
    }
}
LaunchedEffect\(selectedReaderArticleId\) {
    if \(selectedReaderArticleId == null\) {
        inlineReaderRestoredArticleId = null
        readerFullscreenActive = false
        readerBackProgress = 0f
        readerFullscreenBackProgress = 0f
        readerOpenProgress = 1f
        readerOpenAnimating = false
    } else if \(inlineReaderRestoredArticleId != selectedReaderArticleId\) {
        inlineReaderRestoredArticleId = null
    }
}
val animatedContentChannel = \(selectedContentChannelKey ?: lastContentChannelKey\)?.let { key ->
    contentChannels.firstOrNull { it.key == key }
}
val selectedSource = selectedContentChannel?.source
val pagerTopLevelPage = TopLevelMainPages.getOrElse\(pagerState.currentPage\) {
    MainPage.DASHBOARD
}
val currentTopLevelPage = runCatching { MainPage.valueOf\(currentTopLevelPageName\) }
    .getOrDefault\(pagerTopLevelPage\)
    .takeIf { it in TopLevelMainPages }
    ?: pagerTopLevelPage
val channelReturnPage = runCatching { MainPage.valueOf\(channelReturnPageName\) }
    .getOrDefault\(MainPage.RSS\)
    .takeIf { it in TopLevelMainPages }
    ?: MainPage.RSS
val channelTabSwitchDestination = channelTabSwitchDestinationName
    ?.let { runCatching { MainPage.valueOf\(it\) }.getOrNull\(\) }
    ?.takeIf { it in TopLevelMainPages }
val topLevelNavigationTarget = topLevelNavigationTargetName
    ?.let { runCatching { MainPage.valueOf\(it\) }.getOrNull\(\) }
    ?.takeIf { it in TopLevelMainPages }
val page = if \(selectedContentChannel != null\) MainPage.CHANNEL else currentTopLevelPage
val selectedBottomPage = if \(page == MainPage.CHANNEL\) channelReturnPage else currentTopLevelPage

fun returnToDashboard\(
    usePager: Boolean = !isMediumOrExpandedLayout,
    suppressTopLevelTransition: Boolean = false,
    keepPredictiveTransformUntilSettled: Boolean = false
\) {
    if \(suppressTopLevelTransition\) {
        suppressNextTopLevelTransition = true
    }
    if \(keepPredictiveTransformUntilSettled\) {
        tabBackActive = true
        tabBackProgress = 1f
    } else {
        tabBackActive = false
```

```
        tabBackProgress = 0f
    }
    currentTopLevelPageName = MainPage.DASHBOARD.name
    selectedContentChannelKey = null
    selectedReaderArticleId = null
    readerFullscreenActive = false
    readerFullscreenBackProgress = 0f
    readerOpenProgress = 1f
    readerOpenAnimating = false
    readerLeftPaneReturnState = null
    if \(usePager\) {
        coroutineScope.launch {
            pagerState.scrollToPage\(MainPage.DASHBOARD.topLevelIndex\(\)\)
        }
    }
    if \(keepPredictiveTransformUntilSettled\) {
        coroutineScope.launch {
            delay\(64L\)
            tabBackActive = false
            tabBackProgress = 0f
        }
    }
}

fun navigateToTopLevelPage\(
    destination: MainPage,
    usePager: Boolean = !isMediumOrExpandedLayout
\) {
    tabBackActive = false
    tabBackProgress = 0f
    topLevelNavigationTargetName = destination.name
    currentTopLevelPageName = destination.name
    selectedContentChannelKey = null
    selectedReaderArticleId = null
    readerFullscreenActive = false
    readerFullscreenBackProgress = 0f
    readerOpenProgress = 1f
    readerOpenAnimating = false
    readerLeftPaneReturnState = null
    if \(usePager\) {
        coroutineScope.launch {
            pagerState.animateScrollToPage\(destination.topLevelIndex\(\)\)
        }
    }
    coroutineScope.launch {
        delay\(CHANNEL_TRANSITION_MS.toLong\(\)\)
        if \(destination != MainPage.RSS\) {
            selectedContentChannelKey = null
            selectedReaderArticleId = null
            readerFullscreenActive = false
            readerFullscreenBackProgress = 0f
            readerOpenProgress = 1f
            readerOpenAnimating = false
            readerLeftPaneReturnState = null
        }
```

```
        if \(topLevelNavigationTargetName == destination.name\) {
            topLevelNavigationTargetName = null
        }
    }
}

fun navigateToContentChannel\(channelKey: String, returnPage: MainPage = currentTopLevelPage\) {
    val resolvedChannelKey = when \(channelKey\) {
        CONTENT_CHANNEL_IMPORTED_TEXT -> importedContentChannelKey
        else -> channelKey
    }
    channelReturnPageName = returnPage.name
    if \(returnPage in TopLevelMainPages\) {
        currentTopLevelPageName = returnPage.name
    }
    selectedContentChannelKey = resolvedChannelKey
    selectedReaderArticleId = null
    readerFullscreenActive = false
    readerFullscreenBackProgress = 0f
    readerOpenProgress = 1f
    readerOpenAnimating = false
    readerLeftPaneReturnState = null
}

fun switchChannelToTopLevelPage\(destination: MainPage\) {
    if \(channelTabSwitchDestinationName != null || destination !in TopLevelMainPages\) return
    val sourceIndex = channelReturnPage.topLevelIndex\(\)
    val destinationIndex = destination.topLevelIndex\(\)
    if \(destinationIndex == sourceIndex\) {
        navigateToTopLevelPage\(destination\)
        return
    }
    channelTabSwitchDestinationName = destination.name
    channelTabSwitchDirection = if \(destinationIndex > sourceIndex\) 1f else -1f
    channelTabSwitchProgress = 0f
    coroutineScope.launch {
        animate\(
            initialValue = 0f,
            targetValue = 1f,
            animationSpec = tween\(CHANNEL_TRANSITION_MS\)
        \) { value, _ ->
            channelTabSwitchProgress = value
        }
        suppressNextChannelExit = true
        selectedContentChannelKey = null
        currentTopLevelPageName = destination.name
        if \(!isMediumOrExpandedLayout\) {
            pagerState.scrollToPage\(destinationIndex\)
        }
        delay\(32L\)
        channelTabSwitchDestinationName = null
        channelTabSwitchProgress = 0f
        suppressNextChannelExit = false
    }
}
```

```
PredictiveBackHandler\(
    enabled = !isMediumOrExpandedLayout &&
        page == MainPage.CHANNEL &&
        selectedReaderArticle == null
\) { backEvents ->
    try {
        backEvents.collect { backEvent ->
            channelBackProgress = backEvent.progress.coerceIn\(0f, 1f\)
        }
        animate\(
            initialValue = channelBackProgress,
            targetValue = CHANNEL_PREDICTIVE_EXIT_PROGRESS,
            animationSpec = tween\(CHANNEL_PREDICTIVE_EXIT_MS\)
        \) { value, _ ->
            channelBackProgress = value
        }
        selectedContentChannelKey = null
        currentTopLevelPageName = channelReturnPage.name
        if \(!isMediumOrExpandedLayout\) {
            coroutineScope.launch {
                pagerState.animateScrollToPage\(channelReturnPage.topLevelIndex\(\)\)
            }
        }
        delay\(CHANNEL_TRANSITION_MS.toLong\(\) + 32L\)
        if \(selectedContentChannelKey == null\) {
            channelBackProgress = 0f
        }
    } catch \(exception: CancellationException\) {
        animate\(
            initialValue = channelBackProgress,
            targetValue = 0f,
            animationSpec = tween\(CHANNEL_TRANSITION_MS\)
        \) { value, _ ->
            channelBackProgress = value
        }
    }
}

val topLevelBackEnabled = selectedReaderArticle == null &&
    if \(isMediumOrExpandedLayout\) {
        currentTopLevelPage != MainPage.DASHBOARD
    } else {
        page != MainPage.DASHBOARD && page != MainPage.CHANNEL
}
PredictiveBackHandler\(enabled = topLevelBackEnabled\) { backEvents ->
    var resetBackPreviewInFinally = true
    try {
        tabBackActive = true
        tabBackProgress = 0f
        backEvents.collect { backEvent ->
            tabBackProgress = backEvent.progress.coerceIn\(0f, 1f\)
        }
        animate\(
            initialValue = tabBackProgress,
```

```
            targetValue = TAB_PREDICTIVE_EXIT_PROGRESS,
            animationSpec = tween\(TAB_PREDICTIVE_EXIT_MS\)
        \) { value, _ ->
            tabBackProgress = value
        }
        resetBackPreviewInFinally = !isMediumOrExpandedLayout
        returnToDashboard\(
            usePager = !isMediumOrExpandedLayout,
            suppressTopLevelTransition = isMediumOrExpandedLayout,
            keepPredictiveTransformUntilSettled = isMediumOrExpandedLayout
        \)
    } catch \(exception: CancellationException\) {
        animate\(
            initialValue = tabBackProgress,
            targetValue = 0f,
            animationSpec = tween\(CHANNEL_TRANSITION_MS\)
        \) { value, _ ->
            tabBackProgress = value
        }
    } finally {
        if \(resetBackPreviewInFinally\) {
            tabBackActive = false
            tabBackProgress = 0f
        }
    }
}

AdaptiveWindowScope\(modifier = Modifier.fillMaxSize\(\)\) { windowInfo ->
    SideEffect {
        isMediumOrExpandedLayout = windowInfo.isMediumOrExpanded
    }

    LaunchedEffect\(windowInfo.isMediumOrExpanded, pagerTopLevelPage\) {
        if \(!windowInfo.isMediumOrExpanded\) {
            currentTopLevelPageName = pagerTopLevelPage.name
        }
    }
    LaunchedEffect\(currentTopLevelPage, suppressNextTopLevelTransition\) {
        if \(suppressNextTopLevelTransition\) {
            delay\(32L\)
            suppressNextTopLevelTransition = false
        }
    }

    fun openAdaptiveContentChannel\(channelKey: String, returnPage: MainPage\) {
        navigateToContentChannel\(channelKey, returnPage\)
    }

    fun openInlineReaderWithMotion\(articleId: String\) {
        readerLeftPaneReturnState = null
        readerBackProgress = 0f
        readerFullscreenActive = false
        readerFullscreenBackProgress = 0f
        if \(selectedReaderArticleId != null\) {
            selectedReaderArticleId = articleId
```

```
        if \(!readerOpenAnimating\) {
            readerOpenProgress = 1f
        }
        return
    }
    readerOpenProgress = 0f
    readerOpenAnimating = true
    selectedReaderArticleId = articleId
    coroutineScope.launch {
        try {
            animate\(
                initialValue = 0f,
                targetValue = 1f,
                animationSpec = tween\(
                    durationMillis = READER_LEFT_PANE_RETURN_TRANSITION_MS,
                    easing = FastOutSlowInEasing
                \)
            \) { value, _ ->
                readerOpenProgress = value
            }
        } finally {
            readerOpenProgress = 1f
            readerOpenAnimating = false
        }
    }
}

fun openAdaptiveArticle\(article: PhoneArticleEntity\) {
    if \(windowInfo.isMediumOrExpanded && canOpenArticleInline\(article\)\) {
        openInlineReaderWithMotion\(article.articleId\)
    } else {
        onOpenArticle\(article\)
    }
}

fun openIndependentArticleFromImports\(article: PhoneArticleEntity\) {
    if \(windowInfo.isMediumOrExpanded && canOpenArticleInline\(article\)\) {
        channelReturnPageName = MainPage.IMPORTS.name
        selectedContentChannelKey = CONTENT_CHANNEL_INDEPENDENT
        openInlineReaderWithMotion\(article.articleId\)
    } else {
        onOpenArticle\(article\)
    }
}

fun startReaderLeftPaneReturnTransition\(initialProgress: Float = 0f\) {
    val channelKey = selectedContentChannelKey
    val articleId = selectedReaderArticleId
    if \(
        windowInfo.isMediumOrExpanded &&
        channelKey != null &&
        articleId != null &&
        currentTopLevelPage in TopLevelMainPages
    \) {
        readerLeftPaneReturnState = ReaderLeftPaneReturnState\(
```

```
            channelKey = channelKey,
            articleId = articleId,
            returnPage = currentTopLevelPage,
            initialProgress = initialProgress.coerceIn\(0f, 1f\)
        \)
    }
}

fun handleInlineReaderBack\(returnMotionProgress: Float = 0f\) {
    if \(readerFullscreenActive && windowInfo.isMediumOrExpanded\) {
        readerFullscreenActive = false
        readerBackProgress = 0f
        readerFullscreenBackProgress = 0f
    } else if \(
        returnMotionProgress <= 0f &&
        windowInfo.isMediumOrExpanded &&
        selectedReaderArticleId != null &&
        selectedContentChannelKey != null
    \) {
        if \(readerBackAnimating\) return
        readerBackAnimating = true
        coroutineScope.launch {
            try {
                animate\(
                    initialValue = readerBackProgress.coerceIn\(0f, 1f\),
                    targetValue = 1f,
                    animationSpec = tween\(
                        durationMillis = READER_LEFT_PANE_RETURN_TRANSITION_MS,
                        easing = FastOutSlowInEasing
                    \)
                \) { value, _ ->
                    readerBackProgress = value
                }
                startReaderLeftPaneReturnTransition\(1f\)
                selectedReaderArticleId = null
                readerFullscreenActive = false
                readerFullscreenBackProgress = 0f
                readerOpenProgress = 1f
                readerOpenAnimating = false
            } finally {
                readerBackProgress = 0f
                readerBackAnimating = false
            }
        }
    } else {
        startReaderLeftPaneReturnTransition\(returnMotionProgress\)
        selectedReaderArticleId = null
        readerFullscreenActive = false
        readerBackProgress = 0f
        readerFullscreenBackProgress = 0f
        readerOpenProgress = 1f
        readerOpenAnimating = false
    }
}
```

```
PredictiveBackHandler\(enabled = selectedReaderArticle != null\) { backEvents ->
    val fullscreenReaderBack = windowInfo.isMediumOrExpanded && readerFullscreenActive
    try {
        if \(fullscreenReaderBack\) {
            readerBackProgress = 0f
            readerFullscreenBackProgress = 0f
            backEvents.collect { backEvent ->
                readerFullscreenBackProgress = backEvent.progress.coerceIn\(0f, 1f\)
            }
            val remainingDuration = \(READER_FULLSCREEN_BACK_SETTLE_MS *
                \(1f - readerFullscreenBackProgress.coerceIn\(0f, 1f\)\)\)
                .toInt\(\)
                .coerceAtLeast\(1\)
            animate\(
                initialValue = readerFullscreenBackProgress,
                targetValue = 1f,
                animationSpec = tween\(
                    durationMillis = remainingDuration,
                    easing = FastOutSlowInEasing
                \)
            \) { value, _ ->
                readerFullscreenBackProgress = value
            }
            readerFullscreenActive = false
            delay\(READER_FULLSCREEN_BACK_SETTLE_MS.toLong\(\) + 32L\)
            if \(!readerFullscreenActive\) {
                readerFullscreenBackProgress = 0f
            }
        } else {
            readerBackProgress = 0f
            backEvents.collect { backEvent ->
                readerBackProgress = backEvent.progress.coerceIn\(0f, 1f\)
            }
            val splitReaderBack = windowInfo.isMediumOrExpanded
            val targetProgress = if \(windowInfo.isMediumOrExpanded\) 1f else PREDICTIVE_BACK_EXIT_PROGRESS
            val exitDurationMs = if \(splitReaderBack\) {
                READER_LEFT_PANE_RETURN_TRANSITION_MS
            } else {
                PREDICTIVE_BACK_EXIT_ANIMATION_MS
            }
            animate\(
                initialValue = readerBackProgress,
                targetValue = targetProgress,
                animationSpec = tween\(
                    durationMillis = exitDurationMs,
                    easing = FastOutSlowInEasing
                \)
            \) { value, _ ->
                readerBackProgress = value
            }
            val returnMotionProgress = if \(splitReaderBack\) {
                readerBackProgress.coerceIn\(0f, 1f\)
            } else {
                0f
            }
```

```
            handleInlineReaderBack\(returnMotionProgress\)
        }
    } catch \(exception: CancellationException\) {
        if \(fullscreenReaderBack\) {
            animate\(
                initialValue = readerFullscreenBackProgress,
                targetValue = 0f,
                animationSpec = tween\(PREDICTIVE_BACK_CANCEL_ANIMATION_MS\)
            \) { value, _ ->
                readerFullscreenBackProgress = value
            }
        } else {
            animate\(
                initialValue = readerBackProgress,
                targetValue = 0f,
                animationSpec = tween\(PREDICTIVE_BACK_CANCEL_ANIMATION_MS\)
            \) { value, _ ->
                readerBackProgress = value
            }
        }
    }
}

LaunchedEffect\(
    windowInfo.isMediumOrExpanded,
    currentTopLevelPage,
    channelReturnPage,
    selectedContentChannelKey,
    contentChannels
\) {
    val fallbackChannel = if \(
        windowInfo.isMediumOrExpanded &&
        currentTopLevelPage == MainPage.RSS &&
        selectedContentChannel == null
    \) {
        contentChannels.firstOrNull { it.articleCount > 0 }
            ?: contentChannels.firstOrNull\(\)
    } else {
        null
    }
    fallbackChannel?.let { channel ->
        channelReturnPageName = MainPage.RSS.name
        selectedContentChannelKey = channel.key
    }
}

LaunchedEffect\(
    windowInfo.isMediumOrExpanded,
    currentTopLevelPage,
    channelReturnPage,
    selectedContentChannelKey
\) {
    if \(
        windowInfo.isMediumOrExpanded &&
        selectedContentChannelKey != null &&
```

```
        currentTopLevelPage != channelReturnPage
    \) {
        delay\(CHANNEL_TRANSITION_MS.toLong\(\) + 80L\)
        selectedContentChannelKey = null
        selectedReaderArticleId = null
        readerFullscreenActive = false
        readerOpenProgress = 1f
        readerOpenAnimating = false
    }
}

val fabPage = when {
    windowInfo.isMediumOrExpanded && selectedReaderArticle != null -> MainPage.IMPORTS
    windowInfo.isMediumOrExpanded && page == MainPage.CHANNEL -> channelReturnPage
    else -> page
}
val topBarPage = if \(selectedContentChannel == null\) {
    topLevelNavigationTarget ?: page
} else {
    page
}
val navigationSelectedPage = topLevelNavigationTarget ?: if \(windowInfo.isMediumOrExpanded\) {
    currentTopLevelPage
} else {
    selectedBottomPage
}
val readerTakesCurrentPage = selectedReaderArticle != null &&
    \(readerFullscreenActive || !windowInfo.isMediumOrExpanded\)
val readingSplitActive = windowInfo.isMediumOrExpanded && selectedReaderArticle != null
val usesPaneTopBars = windowInfo.isMediumOrExpanded &&
    \(topBarPage == MainPage.RSS || topBarPage == MainPage.IMPORTS || topBarPage == MainPage.CHANNEL\)
val showGlobalTopBar = !readingSplitActive && !readerTakesCurrentPage && !usesPaneTopBars
val showScaffoldTopBar = showGlobalTopBar && windowInfo.navigationType != AdaptiveNavigationType.Rail
val inlineReaderListState = remember\(selectedReaderArticle?.articleId\) {
    LazyListState\(\)
}
val inlineReaderContentNodesCache = remember {
    mutableMapOf<ArticleContentNodesKey, ArticleContentNodesSnapshot>\(\)
}
val movableInlineReaderPane = remember\(
    selectedReaderArticle?.articleId,
    inlineReaderContentReady,
    hydratedSelectedReaderArticle?.articleId != null
\) {
    movableContentOf<InlineReaderPaneInput> { input ->
        InlineArticleReaderPane\(
            article = input.article,
            readerArticle = input.readerArticle,
            importedTextReader = input.importedTextReader,
            onLoadImportedTextChunk = input.onLoadImportedTextChunk,
            onSaveReadingProgress = input.onSaveReadingProgress,
            onBack = input.onBack,
            onOpenImportedArticle = input.onOpenImportedArticle,
            onOpenOriginal = input.onOpenOriginal,
            listState = input.listState,
```

```
            contentReady = input.contentReady,
            contentNodesCache = input.contentNodesCache,
            positionAlreadyRestored = input.positionAlreadyRestored,
            onPositionRestored = input.onPositionRestored,
            fullscreen = input.fullscreen,
            showFullscreenControl = input.showFullscreenControl,
            onToggleFullscreen = input.onToggleFullscreen
        \)
    }
}
val activeInlineReaderPane: @Composable \(Boolean\) -> Unit = { fullscreen ->
    val article = selectedReaderArticleListItem ?: selectedReaderArticle
    if \(article != null\) {
        movableInlineReaderPane\(
            InlineReaderPaneInput\(
                article = article,
                readerArticle = hydratedSelectedReaderArticle
                    ?.takeIf { inlineReaderContentReady && it.articleId == article.articleId },
                importedTextReader = selectedImportedTextReader,
                onLoadImportedTextChunk = onLoadImportedTextChunkForInlineReader,
                onSaveReadingProgress = { progress ->
                    onSaveArticleReadingProgress\(article.articleId, progress\)
                },
                onBack = { handleInlineReaderBack\(\) },
                onOpenImportedArticle = { url -> uriHandler.openUri\(url\) },
                onOpenOriginal = { url -> uriHandler.openUri\(url\) },
                listState = inlineReaderListState,
                contentReady = inlineReaderContentReady,
                contentNodesCache = inlineReaderContentNodesCache,
                positionAlreadyRestored = inlineReaderRestoredArticleId == article.articleId,
                onPositionRestored = { restoredArticleId ->
                    if \(selectedReaderArticleId == restoredArticleId\) {
                        inlineReaderRestoredArticleId = restoredArticleId
                    }
                },
                fullscreen = fullscreen,
                showFullscreenControl = windowInfo.isMediumOrExpanded,
                onToggleFullscreen = {
                    readerFullscreenBackProgress = 0f
                    readerFullscreenActive = !readerFullscreenActive
                }
            \)
        \)
    }
}
@Composable
fun RenderGlobalTopBar\(modifier: Modifier = Modifier\) {
    MainTopBar\(
        page = if \(windowInfo.isMediumOrExpanded && topBarPage == MainPage.CHANNEL\) channelReturnPage else topBarPage,
        selectedChannel = if \(windowInfo.isMediumOrExpanded\) null else selectedContentChannel,
        canRefreshRss = uiState.rssSources.any { !ImportedContentIds.isImportedContentUrl\(it.url\) } && !uiState.isBusy,
        canRefreshSource = !windowInfo.isMediumOrExpanded &&
            selectedContentChannel?.canRefresh == true &&
            selectedSource != null &&
            selectedSource.url !in uiState.refreshingRssSourceUrls &&
```

```
            !uiState.isBusy,
        onBack = {
            navigateToTopLevelPage\(
                channelReturnPage,
                usePager = !windowInfo.isMediumOrExpanded
            \)
        },
        onRefreshAllRssSources = onRefreshAllRssSources,
        onRefreshSelectedSource = {
            if \(selectedContentChannel?.canRefresh == true\) {
                selectedSource?.let\(onRefreshRssSource\)
            }
        },
        onExportBluetoothLog = onExportBluetoothLog,
        onOpenAccount = onOpenAccount,
        onOpenSettings = onOpenSettings,
        showAccountAction = showAccountFeatures,
        modifier = modifier
    \)
}
Scaffold\(
    topBar = {
        if \(showScaffoldTopBar\) {
            RenderGlobalTopBar\(\)
        }
    },
    bottomBar = {
        if \(windowInfo.navigationType == AdaptiveNavigationType.BottomBar && selectedReaderArticle == null\) {
            MainNavigationBar\(
                selectedPage = navigationSelectedPage,
                onSelectPage = { destination ->
                    if \(page == MainPage.CHANNEL && destination != channelReturnPage\) {
                        switchChannelToTopLevelPage\(destination\)
                    } else {
                        navigateToTopLevelPage\(
                            destination,
                            usePager = !windowInfo.isMediumOrExpanded
                        \)
                    }
                }
            \)
        }
    },
    floatingActionButton = {
        if \(selectedReaderArticle == null\) {
            MainFloatingActionButton\(
                page = fabPage,
                isBusy = uiState.isBusy,
                selectedSource = if \(windowInfo.isMediumOrExpanded\) null else selectedSource,
                selectedSourceRefreshing = selectedSource?.url?.let { it in uiState.refreshingRssSourceUrls } == true,
                canRefreshSelectedSource = !windowInfo.isMediumOrExpanded && selectedContentChannel?.canRefresh == true,
                onSyncLibrary = onSyncLibrary,
                onAddRssSource = { urlDialogMode = UrlDialogMode.RSS },
                onRefreshSelectedSource = {
                    if \(selectedContentChannel?.canRefresh == true\) {
```

```
                selectedSource?.let\(onRefreshRssSource\)
            }
        }
    \)
  }
 }
\) { contentPadding ->
 Row\(modifier = Modifier.fillMaxSize\(\)\) {
  if \(windowInfo.navigationType == AdaptiveNavigationType.Rail\) {
   MainNavigationRail\(
    selectedPage = navigationSelectedPage,
    contentPadding = contentPadding,
    onSelectPage = { destination ->
     if \(!windowInfo.isMediumOrExpanded && page == MainPage.CHANNEL && destination != channelReturnPage\) {
      switchChannelToTopLevelPage\(destination\)
     } else {
      navigateToTopLevelPage\(
       destination,
       usePager = !windowInfo.isMediumOrExpanded
      \)
     }
    }
   \)
   VerticalDivider\(
    modifier = Modifier
     .zIndex\(2f\)
     .fillMaxHeight\(\)
     .padding\(
      top = contentPadding.calculateTopPadding\(\),
      bottom = contentPadding.calculateBottomPadding\(\)
     \)
   \)
  }
  Column\(
   modifier = Modifier
    .weight\(1f\)
    .fillMaxSize\(\)
  \) {
   Box\(
    modifier = Modifier
     .weight\(1f\)
     .fillMaxWidth\(\)
   \) {
   channelTabSwitchDestination?.takeUnless { windowInfo.isMediumOrExpanded }?.let { destination ->
    OpaquePageLayer\(
     modifier = Modifier
      .fillMaxSize\(\)
      .channelTabSwitchTargetPreview\(
       progress = channelTabSwitchProgress,
       direction = channelTabSwitchDirection
      \)
      .zIndex\(0.5f\)
    \) {
     when \(destination\) {
      MainPage.DASHBOARD -> DashboardPage\(
```

```
import androidx.room.PrimaryKey

@Entity\(
    tableName = "phone_rss_sources",
    indices = [
        Index\(value = ["deleted", "sortOrder"]\)
    ]
\)
data class PhoneRssSourceEntity\(
    @PrimaryKey val url: String,
    val sourceDeviceId: String,
    val title: String,
    val description: String,
    val siteUrl: String?,
    val imageUrl: String?,
    val createdAt: Long,
    val updatedAt: Long,
    val sortOrder: Long,
    val isPinned: Boolean,
    val deleted: Boolean,
    val deletedAt: Long
\)
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/data/db/PhoneSavedItemDao.kt =====
package com.lightningstudio.watchrss.phone.data.db

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import kotlinx.coroutines.flow.Flow

@Dao
interface PhoneSavedItemDao {
    @Query\("SELECT * FROM phone_saved_items WHERE type = :type ORDER BY syncedAt DESC, title ASC"\)
    fun observeByType\(type: String\): Flow<List<PhoneSavedItemEntity>>

    @Query\("DELETE FROM phone_saved_items WHERE type = :type"\)
    suspend fun deleteByType\(type: String\)

    @Insert\(onConflict = OnConflictStrategy.REPLACE\)
    suspend fun upsertAll\(items: List<PhoneSavedItemEntity>\)

    @Query\("SELECT * FROM phone_saved_items"\)
    suspend fun getAll\(\): List<PhoneSavedItemEntity>

    @Query\("DELETE FROM phone_saved_items"\)
    suspend fun deleteAll\(\)
}
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/data/db/PhoneSavedItemEntity.kt =====
package com.lightningstudio.watchrss.phone.data.db

import androidx.room.Entity

@Entity\(
    tableName = "phone_saved_items",
```

```
    primaryKeys = ["type", "stableKey"]
\)
data class PhoneSavedItemEntity\(
    val type: String,
    val stableKey: String,
    val remoteId: Long,
    val title: String,
    val link: String,
    val summary: String,
    val channelTitle: String,
    val pubDate: String,
    val syncedAt: Long
\)
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/data/db/SyncChangeLogDao.kt =====
package com.lightningstudio.watchrss.phone.data.db

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.Query

@Dao
interface SyncChangeLogDao {
    @Insert
    suspend fun insert\(change: SyncChangeLogEntity\): Long

    @Query\("SELECT COALESCE\(MAX\(seq\), 0\) FROM sync_change_log"\)
    suspend fun maxSeq\(\): Long

    @Query\(
        """
        SELECT entityId
        FROM sync_change_log
        WHERE kind = :kind AND seq > :afterSeq
        GROUP BY entityId
        ORDER BY MIN\(seq\) ASC
        """
    \)
    suspend fun entityIdsChangedAfter\(kind: String, afterSeq: Long\): List<String>

    @Query\(
        """
        SELECT entityId, MAX\(changedAt\) AS changedAt
        FROM sync_change_log
        WHERE kind = :kind AND entityId IN \(:entityIds\)
        GROUP BY entityId
        """
    \)
    suspend fun maxChangedAtByEntityIds\(kind: String, entityIds: List<String>\): List<SyncChangeLogEntityState>

    @Query\("DELETE FROM sync_change_log"\)
    suspend fun deleteAll\(\)
}

data class SyncChangeLogEntityState\(
    val entityId: String,
```

```
    val changedAt: Long
\)
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/data/db/SyncChangeLogEntity.kt =====
package com.lightningstudio.watchrss.phone.data.db

import androidx.room.Entity
import androidx.room.Index
import androidx.room.PrimaryKey

@Entity\(
    tableName = "sync_change_log",
    indices = [
        Index\(value = ["kind", "entityId"]\),
        Index\(value = ["seq"]\)
    ]
\)
data class SyncChangeLogEntity\(
    @PrimaryKey\(autoGenerate = true\) val seq: Long = 0L,
    val kind: String,
    val entityId: String,
    val changedAt: Long,
    val originDeviceId: String,
    val reason: String,
    val createdAt: Long
\)
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/data/db/SyncPeerStateDao.kt =====
package com.lightningstudio.watchrss.phone.data.db

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query

@Dao
interface SyncPeerStateDao {
    @Query\("SELECT * FROM sync_peer_state WHERE peerDeviceId = :peerDeviceId LIMIT 1"\)
    suspend fun get\(peerDeviceId: String\): SyncPeerStateEntity?

    @Insert\(onConflict = OnConflictStrategy.REPLACE\)
    suspend fun upsert\(state: SyncPeerStateEntity\)

    @Query\("DELETE FROM sync_peer_state"\)
    suspend fun deleteAll\(\)
}
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/data/db/SyncPeerStateEntity.kt =====
package com.lightningstudio.watchrss.phone.data.db

import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity\(tableName = "sync_peer_state"\)
data class SyncPeerStateEntity\(
    @PrimaryKey val peerDeviceId: String,
    val lastLocalSeqAckedByPeer: Long = 0L,
    val lastRemoteSeqApplied: Long = 0L,
```

```
    val lastFullSyncAt: Long = 0L,
    val lastProtocolVersion: Int = 0,
    val updatedAt: Long = 0L
\)
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/data/local/ArticleContentStore.kt =====
package com.lightningstudio.watchrss.phone.data.local

import android.content.Context
import java.io.File
import java.io.RandomAccessFile
import kotlin.math.ceil

interface ArticleContentStore {
    fun markerFor\(articleId: String\): String
    fun isMarker\(value: String\): Boolean
    fun storeText\(articleId: String, text: String\): String
    fun loadText\(marker: String\): String?
    fun textChunkHandle\(marker: String, chunkBytes: Int = ARTICLE_TEXT_CHUNK_BYTES\): StoredTextChunkHandle?
    fun loadTextChunk\(marker: String, chunkIndex: Int, chunkBytes: Int = ARTICLE_TEXT_CHUNK_BYTES\): String?
    fun prune\(retainedMarkers: Set<String>\) = Unit
}

data class StoredTextChunkHandle\(
    val marker: String,
    val byteLength: Long,
    val chunkBytes: Int,
    val chunkCount: Int
\)

class FileArticleContentStore\(context: Context\) : ArticleContentStore {
    private val directory = File\(context.applicationContext.filesDir, "imported_text"\)

    override fun markerFor\(articleId: String\): String {
        return "$ARTICLE_CONTENT_MARKER_PREFIX${safeFileName\(articleId\)}.txt"
    }

    override fun isMarker\(value: String\): Boolean {
        return isArticleContentMarker\(value\)
    }

    override fun storeText\(articleId: String, text: String\): String {
        if \(!directory.exists\(\)\) {
            directory.mkdirs\(\)
        }
        val marker = markerFor\(articleId\)
        File\(directory, marker.removePrefix\(ARTICLE_CONTENT_MARKER_PREFIX\)\).writeText\(text, Charsets.UTF_8\)
        return marker
    }

    override fun loadText\(marker: String\): String? {
        if \(!isMarker\(marker\)\) return null
        val fileName = marker.removePrefix\(ARTICLE_CONTENT_MARKER_PREFIX\)
        return runCatching {
            File\(directory, fileName\).takeIf { it.isFile }?.readText\(Charsets.UTF_8\)
        }.getOrNull\(\)
```

```
}

override fun textChunkHandle\(marker: String, chunkBytes: Int\): StoredTextChunkHandle? {
    if \(chunkBytes <= 0\) return null
    val file = fileFor\(marker\)?.takeIf { it.isFile } ?: return null
    val byteLength = file.length\(\)
    return StoredTextChunkHandle\(
        marker = marker,
        byteLength = byteLength,
        chunkBytes = chunkBytes,
        chunkCount = ceil\(byteLength.toDouble\(\) / chunkBytes.toDouble\(\)\).toInt\(\).coerceAtLeast\(1\)
    \)
}

override fun loadTextChunk\(marker: String, chunkIndex: Int, chunkBytes: Int\): String? {
    if \(chunkIndex < 0 || chunkBytes <= 0\) return null
    val file = fileFor\(marker\)?.takeIf { it.isFile } ?: return null
    return runCatching {
        RandomAccessFile\(file, "r"\).use { reader ->
            val byteLength = reader.length\(\)
            val nominalStart = chunkIndex.toLong\(\) * chunkBytes
            if \(nominalStart >= byteLength\) return@use null
            val nominalEnd = \(nominalStart + chunkBytes\).coerceAtMost\(byteLength\)
            val start = reader.adjustUtf8Boundary\(nominalStart, byteLength\)
            val end = reader.adjustUtf8Boundary\(nominalEnd, byteLength\)
            if \(end <= start\) return@use ""
            val bytes = ByteArray\(\(end - start\).toInt\(\)\)
            reader.seek\(start\)
            reader.readFully\(bytes\)
            String\(bytes, Charsets.UTF_8\)
        }
    }.getOrNull\(\)
}

override fun prune\(retainedMarkers: Set<String>\) {
    if \(!directory.isDirectory\) return
    val retainedFileNames = retainedMarkers
        .asSequence\(\)
        .filter\(::isMarker\)
        .map { it.removePrefix\(ARTICLE_CONTENT_MARKER_PREFIX\) }
        .toSet\(\)
    directory.listFiles\(\)?.forEach { file ->
        if \(file.isFile && file.name !in retainedFileNames\) {
            runCatching { file.delete\(\) }
        }
    }
}

private fun RandomAccessFile.adjustUtf8Boundary\(requested: Long, byteLength: Long\): Long {
    var position = requested.coerceIn\(0L, byteLength\)
    if \(position <= 0L || position >= byteLength\) return position
    seek\(position\)
    var value = read\(\)
    while \(position > 0L && value >= 0 && \(value and UTF8_CONTINUATION_MASK\) == UTF8_CONTINUATION_PREFIX\) {
        position -= 1L
```

```
            seek\(position\)
            value = read\(\)
        }
        return position
    }

    private fun fileFor\(marker: String\): File? {
        if \(!isMarker\(marker\)\) return null
        val fileName = marker.removePrefix\(ARTICLE_CONTENT_MARKER_PREFIX\)
        return File\(directory, fileName\)
    }

    private fun safeFileName\(articleId: String\): String {
        return articleId.replace\(Regex\("""[^A-Za-z0-9._-]"""\), "_"\)
            .take\(MAX_FILE_NAME_CHARS\)
            .ifBlank { "article" }
    }

    companion object {
        private const val MAX_FILE_NAME_CHARS = 96
        private const val UTF8_CONTINUATION_MASK = 0xC0
        private const val UTF8_CONTINUATION_PREFIX = 0x80
    }
}

fun isArticleContentMarker\(value: String?\): Boolean {
    return value?.startsWith\(ARTICLE_CONTENT_MARKER_PREFIX\) == true
}

const val ARTICLE_CONTENT_MARKER_PREFIX = "watchrss-local-text:"
const val ARTICLE_TEXT_CHUNK_BYTES = 2 * 1024
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/data/local/PhoneDeviceIdentity.kt =====
package com.lightningstudio.watchrss.phone.data.local

import android.content.Context
import java.util.UUID

class PhoneDeviceIdentity\(context: Context\) {
    private val preferences = context.applicationContext.getSharedPreferences\(
        "phone_device_identity",
        Context.MODE_PRIVATE
    \)

    val deviceId: String
        get\(\) {
            val existing = preferences.getString\(KEY_DEVICE_ID, null\)
            if \(!existing.isNullOrBlank\(\)\) return existing
            val created = UUID.randomUUID\(\).toString\(\)
            preferences.edit\(\).putString\(KEY_DEVICE_ID, created\).apply\(\)
            return created
        }

    companion object {
        private const val KEY_DEVICE_ID = "device_id"
    }
```

```
}
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/data/log/BluetoothDebugLog.kt =====
package com.lightningstudio.watchrss.phone.data.log

import android.content.ContentResolver
import android.content.Context
import android.net.Uri
import android.os.Build
import android.util.Log
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import java.io.File
import java.time.Instant
import java.util.concurrent.atomic.AtomicInteger

class BluetoothDebugLog\(context: Context\) {
    private val appContext = context.applicationContext
    private val lock = Any\(\)
    private val file: File
        get\(\) = File\(appContext.filesDir, FILE_NAME\)

    fun append\(message: String, throwable: Throwable? = null\) {
        appendEvent\(
            event = "message",
            fields = mapOf\("text" to message\),
            throwable = throwable
        \)
    }

    fun appendEvent\(
        event: String,
        sessionId: String? = null,
        fields: Map<String, Any?> = emptyMap\(\),
        throwable: Throwable? = null
    \) {
        val entry = buildString {
            append\(Instant.now\(\).toString\(\)\)
            append\(" event="\)
            append\(event.escapeValue\(\)\)
            if \(!sessionId.isNullOrBlank\(\)\) {
                append\(" session="\)
                append\(sessionId.escapeValue\(\)\)
            }
            fields.forEach { \(key, value\) ->
                append\(' '\)
                append\(key.filter { it.isLetterOrDigit\(\) || it == '_' || it == '-' }\)
                append\('='\)
                append\(value.formatValue\(\)\)
            }
            appendLine\(\)
            if \(throwable != null\) {
                append\(Log.getStackTraceString\(throwable\)\)
                appendLine\(\)
            }
        }
```

```
    synchronized\(lock\) {
        runCatching {
            file.appendText\(entry\)
            if \(file.length\(\) > MAX_LOG_BYTES\) {
                trimLocked\(\)
            }
        }
    }
}

suspend fun exportTo\(contentResolver: ContentResolver, uri: Uri\): Long =
    withContext\(Dispatchers.IO\) {
        val text = snapshot\(\)
        contentResolver.openOutputStream\(uri\)?.use { output ->
            val bytes = text.toByteArray\(Charsets.UTF_8\)
            output.write\(bytes\)
            bytes.size.toLong\(\)
        } ?: error\("无法创建日志文件"\)
    }

private fun snapshot\(\): String {
    val body = synchronized\(lock\) {
        if \(file.exists\(\)\) {
            file.readText\(\)
        } else {
            ""
        }
    }
    return buildString {
        appendLine\("WatchRSS Phone Bluetooth Debug Log"\)
        appendLine\("exportedAt=${Instant.now\(\)}"\)
        appendLine\("package=${appContext.packageName}"\)
        appendLine\("device=${Build.MANUFACTURER} ${Build.MODEL}"\)
        appendLine\("sdk=${Build.VERSION.SDK_INT}"\)
        appendLine\(\)
        append\(body.ifBlank { "No Bluetooth debug entries recorded." }\)
    }
}

private fun trimLocked\(\) {
    val text = file.readText\(\)
    if \(text.length <= TRIM_TO_CHARS\) return
    file.writeText\(text.takeLast\(TRIM_TO_CHARS\)\)
}

private fun Any?.formatValue\(\): String {
    return when \(this\) {
        null -> "null"
        is Number,
        is Boolean -> toString\(\)
        else -> toString\(\).escapeValue\(\)
    }
}

private fun String.escapeValue\(\): String {
```

```
        val clipped = if \(length > MAX_VALUE_CHARS\) {
            take\(MAX_VALUE_CHARS\) + "...<truncated>"
        } else {
            this
        }
        return buildString {
            append\("'"\)
            clipped.forEach { char ->
                when \(char\) {
                    '\\' -> append\("\\\\"\)
                    '"' -> append\("\\\""\)
                    '\n' -> append\("\\n"\)
                    '\r' -> append\("\\r"\)
                    '\t' -> append\("\\t"\)
                    else -> append\(char\)
                }
            }
            append\("'"\)
        }
    }

    companion object {
        private val sessionCounter = AtomicInteger\(0\)

        fun newSessionId\(prefix: String\): String {
            val safePrefix = prefix.filter { it.isLetterOrDigit\(\) || it == '-' || it == '_' }
                .ifBlank { "bt" }
            return "$safePrefix-${System.currentTimeMillis\(\).toString\(36\)}-${sessionCounter.incrementAndGet\(\)}"
        }

        private const val FILE_NAME = "bluetooth_debug.log"
        private const val MAX_LOG_BYTES = 512 * 1024
        private const val TRIM_TO_CHARS = 256 * 1024
        private const val MAX_VALUE_CHARS = 600
    }
}
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/data/model/ImportedContentIds.kt =====
package com.lightningstudio.watchrss.phone.data.model

object ImportedContentIds {
    const val ROOT_SOURCE_URL = "https://watchrss.local/import-content"
    const val ROOT_SOURCE_TITLE = "导入内容"
    const val EPUB_SOURCE_ROOT_URL = "https://watchrss.local/import-epub"

    fun txtArticleUrl\(contentKey: String\): String {
        return "$ROOT_SOURCE_URL/txt/$contentKey"
    }

    fun epubSourceUrl\(bookKey: String\): String {
        return "$EPUB_SOURCE_ROOT_URL/$bookKey"
    }

    fun epubChapterUrl\(bookKey: String, chapterIndex: Int, chapterKey: String\): String {
        val index = chapterIndex.toString\(\).padStart\(4, '0'\)
        return "${epubSourceUrl\(bookKey\)}/chapter/$index-$chapterKey"
```

```
    }

    fun isImportedContentUrl\(url: String?\): Boolean {
        val normalized = url?.trim\(\)?.lowercase\(\) ?: return false
        return normalized.startsWith\(ROOT_SOURCE_URL\) ||
            normalized.startsWith\(EPUB_SOURCE_ROOT_URL\)
    }

    fun isImportedTextSourceUrl\(url: String?\): Boolean {
        val normalized = url?.trim\(\)?.lowercase\(\) ?: return false
        return normalized == ROOT_SOURCE_URL
    }

    fun isImportedTextArticleUrl\(url: String?\): Boolean {
        val normalized = url?.trim\(\)?.lowercase\(\) ?: return false
        return normalized.startsWith\("$ROOT_SOURCE_URL/txt/"\)
    }

    fun isImportedEpubSourceUrl\(url: String?\): Boolean {
        val normalized = url?.trim\(\)?.lowercase\(\) ?: return false
        return normalized.startsWith\(EPUB_SOURCE_ROOT_URL\) ||
            normalized.startsWith\("$ROOT_SOURCE_URL/epub/"\)
    }
}
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/data/model/PhoneSavedItemType.kt =====
package com.lightningstudio.watchrss.phone.data.model

enum class PhoneSavedItemType\(val wirePath: String, val displayName: String\) {
    FAVORITE\(
        wirePath = "/getFavorites",
        displayName = "收藏"
    \),
    WATCH_LATER\(
        wirePath = "/getWatchlaterList",
        displayName = "稍后再看"
    \)
}
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/data/telemetry/PhoneUsageTelemetry.kt =====
package com.lightningstudio.watchrss.phone.data.telemetry

import android.content.Context
import android.os.Build
import com.lightningstudio.watchrss.phone.BuildConfig
import com.lightningstudio.watchrss.phone.account.AccountEnvironment
import com.lightningstudio.watchrss.phone.account.PhoneAccountRepository
import com.lightningstudio.watchrss.phone.account.PhoneInstallationIdentity
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import okhttp3.MediaType.Companion.toMediaType
import okhttp3.OkHttpClient
import okhttp3.Request
import okhttp3.RequestBody.Companion.toRequestBody
import org.json.JSONObject
import java.util.concurrent.TimeUnit
```

```
class PhoneUsageTelemetry\(
    private val context: Context,
    private val environment: AccountEnvironment,
    private val installationIdentity: PhoneInstallationIdentity,
    private val accountRepository: PhoneAccountRepository,
    private val appScope: CoroutineScope,
    private val httpClient: OkHttpClient = defaultHttpClient\(\)
\) {
    private val preferences = context.applicationContext.getSharedPreferences\(
        "watchrss_usage_telemetry",
        Context.MODE_PRIVATE
    \)

    fun recordAppLaunch\(\) {
        capture\("app_opened"\)
    }

    fun recordScreenOpen\(screen: String\) {
        preferences.edit\(\)
            .putInt\("screen_count_$screen", preferences.getInt\("screen_count_$screen", 0\) + 1\)
            .apply\(\)
        capture\("screen_opened", mapOf\("screen" to screen\)\)
    }

    fun recordScreenDuration\(screen: String, durationMs: Long\) {
        if \(durationMs <= 0L\) return
        preferences.edit\(\)
            .putLong\("screen_duration_$screen", preferences.getLong\("screen_duration_$screen", 0L\) + durationMs\)
            .apply\(\)
        capture\("screen_duration", mapOf\("screen" to screen, "durationMs" to durationMs\)\)
    }

    fun recordSyncResult\(success: Boolean, kind: String, message: String? = null\) {
        capture\(
            event = "sync_completed",
            properties = mapOf\(
                "success" to success,
                "kind" to kind,
                "message" to message.orEmpty\(\)
            \)
        \)
    }

    private fun capture\(event: String, properties: Map<String, Any?> = emptyMap\(\)\) {
        if \(!environment.isTelemetryConfigured\) return
        appScope.launch\(Dispatchers.IO\) {
            runCatching {
                val userId = accountRepository.session.value?.userId
                val distinctId = userId ?: installationIdentity.installId
                val payload = JSONObject\(\).apply {
                    put\("api_key", environment.posthogApiKey\)
                    put\("event", event\)
                    put\("distinct_id", distinctId\)
                    put\("properties", JSONObject\(\).apply {
```

```
                put\("installId", installationIdentity.installId\)
                put\("userId", userId.orEmpty\(\)\)
                put\("platform", "phone"\)
                put\("packageName", context.packageName\)
                put\("appVersionName", BuildConfig.VERSION_NAME\)
                put\("appVersionCode", BuildConfig.VERSION_CODE\)
                put\("firstInstalledAt", installationIdentity.firstInstalledAtMillis\)
                put\("databaseInitializedAt", installationIdentity.databaseInitializedAtMillis\)
                put\("deviceModel", "${Build.MANUFACTURER} ${Build.MODEL}"\)
                put\("sdk", Build.VERSION.SDK_INT\)
                properties.forEach { \(key, value\) -> put\(key, value\) }
              }\)
            }
            val request = Request.Builder\(\)
              .url\("${environment.posthogHost}/capture/"\)
              .post\(payload.toString\(\).toRequestBody\(JSON_MEDIA_TYPE\)\)
              .build\(\)
            httpClient.newCall\(request\).execute\(\).close\(\)
          }
        }
      }

      companion object {
        private val JSON_MEDIA_TYPE = "application/json; charset=utf-8".toMediaType\(\)

        private fun defaultHttpClient\(\): OkHttpClient =
          OkHttpClient.Builder\(\)
            .connectTimeout\(8, TimeUnit.SECONDS\)
            .readTimeout\(12, TimeUnit.SECONDS\)
            .writeTimeout\(12, TimeUnit.SECONDS\)
            .build\(\)
      }
    }

    // ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/platform/OnlineNovelLinkDetector.kt =====
    package com.lightningstudio.watchrss.phone.platform

    import java.net.URI

    object OnlineNovelLinkDetector {
      private val httpUrlPattern = Regex\("""https?://[^\\s<>\\"']+""", RegexOption.IGNORE_CASE\)

      private val onlineNovelDomains = setOf\(
        // 番茄小说 / 番茄畅听
        "fanqienovel.com",
        "changdunovel.com",
        // 七猫小说
        "qimao.com",
        "wtzw.com",
        // 晋江文学城
        "jjwxc.net",
        "jjwxc.com",
        "jjwxc.cn",
        // 其他常见在线小说库
        "qidian.com",
```

```
        "qdmm.com",
        "zongheng.com",
        "17k.com",
        "readnovel.com",
        "hongxiu.com",
        "xxsy.net",
        "ciweimao.com",
        "sfacg.com",
        "faloo.com",
        "shuqi.com",
        "tadu.com",
        "heiyan.com",
        "ruochu.com",
        "ihuaben.com",
        "book.qq.com",
        "yunqi.qq.com",
        "weread.qq.com"
    \)

    fun isOnlineNovelLink\(text: String?\): Boolean = findOnlineNovelUrl\(text\) != null

    fun findOnlineNovelUrl\(text: String?\): String? {
        val candidate = text?.trim\(\).orEmpty\(\)
        if \(candidate.isBlank\(\)\) return null
        return httpUrlPattern.findAll\(candidate\)
            .map { match -> match.value.trimUrlTail\(\) }
            .firstOrNull\(::hasOnlineNovelHost\)
    }

    private fun hasOnlineNovelHost\(url: String\): Boolean {
        val uri = runCatching { URI\(url\) }.getOrNull\(\) ?: return false
        val scheme = uri.scheme?.lowercase\(\).orEmpty\(\)
        if \(scheme != "http" && scheme != "https"\) return false
        val host = uri.host?.lowercase\(\)?.trimEnd\('.'\).orEmpty\(\)
        if \(host.isBlank\(\)\) return false
        return onlineNovelDomains.any { domain ->
            host == domain || host.endsWith\(".$domain"\)
        }
    }

    private fun String.trimUrlTail\(\): String = trimEnd\(
        '.', ',', ' ', '，', '。', ';', '；', '!', '！', '?', '？',
        '\)', '）', ']', '】', '}', '》'
    \)
}
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/platform/PlatformLinkRouter.kt =====
package com.lightningstudio.watchrss.phone.platform

import java.net.URI

enum class PlatformLinkKind\(val displayName: String\) {
    BILI\("B站"\),
    DOUYIN\("抖音"\)
}
```

```
object PlatformLinkRouter {
    fun detect\(url: String?\): PlatformLinkKind? {
        val candidate = url?.trim\(\).orEmpty\(\)
        if \(candidate.isBlank\(\)\) return null
        val uri = runCatching { URI\(candidate\) }.getOrNull\(\) ?: return null
        val scheme = uri.scheme?.lowercase\(\).orEmpty\(\)
        if \(scheme != "http" && scheme != "https"\) return null
        val host = uri.host?.lowercase\(\)?.trimEnd\('.'\).orEmpty\(\)
        if \(host.isBlank\(\)\) return null
        return when {
            host == "b23.tv" || hostMatches\(host, "bilibili.com"\) || hostMatches\(host, "bili2233.cn"\) ->
                PlatformLinkKind.BILI

            hostMatches\(host, "douyin.com"\) || hostMatches\(host, "iesdouyin.com"\) ->
                PlatformLinkKind.DOUYIN

            else -> null
        }
    }

    private fun hostMatches\(host: String, domain: String\): Boolean {
        return host == domain || host.endsWith\(".$domain"\)
    }
}
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/ui/BackdropDemoScreen.kt =====
package com.lightningstudio.watchrss.phone.ui

import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.ui.graphics.Brush
import com.kyant.backdrop.backdrops.*

/**
 * Backdrop 库效果演示页面
 *
 * 展示 kyant/backdrop \(Jetpack Compose Liquid Glass\) 的核心功能：
 * - rememberLayerBackdrop\(\) — 创建层 backdrop
 * - Modifier.layerBackdrop\(\) — 应用 backdrop 到组件
 * - Glassmorphism / 玻璃拟态效果
 */
@OptIn\(ExperimentalMaterial3Api::class\)
```

```
@Composable
fun BackdropDemoScreen\(
    onNavigateBack: \(\) -> Unit
\) {
    val scrollState = rememberScrollState\(\)
    val backgroundColor = MaterialTheme.colorScheme.background

    Scaffold\(
        topBar = {
            TopAppBar\(
                title = { Text\("Backdrop Demo"\) },
                navigationIcon = {
                    IconButton\(onClick = onNavigateBack\) {
                        Icon\(
                            imageVector = Icons.AutoMirrored.Filled.ArrowBack,
                            contentDescription = "返回"
                        \)
                    }
                }
            \)
        }
    \) { padding ->
        Box\(modifier = Modifier.fillMaxSize\(\)\) {
            // 创建 layer backdrop，捕获背后的内容
            val backdrop = rememberLayerBackdrop {
                drawRect\(backgroundColor\)
                drawContent\(\)
            }

            Column\(
                modifier = Modifier
                    .fillMaxSize\(\)
                    .padding\(padding\)
                    .verticalScroll\(scrollState\)
                    .padding\(16.dp\),
                verticalArrangement = Arrangement.spacedBy\(16.dp\)
            \) {
                Text\(
                    text = "Backdrop Library Demo",
                    style = MaterialTheme.typography.headlineMedium,
                    fontWeight = FontWeight.Bold
                \)

                Text\(
                    text = "验证 kyant/backdrop \(io.github.kyant0\) 在 Android_WatchRSS_Phone 中的集成效果。",
                    style = MaterialTheme.typography.bodyMedium,
                    color = MaterialTheme.colorScheme.onSurfaceVariant
                \)

                Spacer\(modifier = Modifier.height\(8.dp\)\)

                // Demo 1: Layer Backdrop
                DemoCard\(title = "Demo 1: Layer Backdrop"\) {
                    Box\(
                        modifier = Modifier
```

```
            .fillMaxWidth\(\)
            .height\(200.dp\)
            .clip\(RoundedCornerShape\(16.dp\)\)
            .background\(MaterialTheme.colorScheme.primaryContainer\)
        \) {
            Box\(
                modifier = Modifier.fillMaxSize\(\),
                contentAlignment = Alignment.Center
            \) {
                Text\(
                    text = "Background Layer",
                    style = MaterialTheme.typography.titleLarge,
                    color = MaterialTheme.colorScheme.onPrimaryContainer
                \)
            }

            // 使用 layerBackdrop 创建玻璃覆盖层
            Box\(
                modifier = Modifier
                    .align\(Alignment.BottomCenter\)
                    .fillMaxWidth\(\)
                    .height\(80.dp\)
                    .layerBackdrop\(backdrop\)
            \) {
                Text\(
                    text = "Glass Overlay \(layerBackdrop\)",
                    modifier = Modifier.align\(Alignment.Center\),
                    style = MaterialTheme.typography.bodyLarge,
                    color = MaterialTheme.colorScheme.onSurface
                \)
            }
        }
    }

    // Demo 2: Glassmorphism
    DemoCard\(title = "Demo 2: Glassmorphism Card"\) {
        Box\(
            modifier = Modifier
                .fillMaxWidth\(\)
                .height\(180.dp\)
                .clip\(RoundedCornerShape\(24.dp\)\)
                .background\(
                    brush = Brush.linearGradient\(
                        colors = listOf\(
                            Color\(0xFF6C63FF\),
                            Color\(0xFF00BFA6\)
                        \)
                    \)
                \),
            contentAlignment = Alignment.Center
        \) {
            Card\(
                modifier = Modifier
                    .padding\(24.dp\)
                    .fillMaxWidth\(0.8f\),
```

```
        shape = RoundedCornerShape\(20.dp\),
        colors = CardDefaults.cardColors\(
          containerColor = Color.White.copy\(alpha = 0.15f\)
        \)
      \) {
        Box\(
          modifier = Modifier.padding\(20.dp\),
          contentAlignment = Alignment.Center
        \) {
          Text\(
            text = "Glass Card \(alpha blur\)",
            style = MaterialTheme.typography.titleMedium,
            color = Color.White
          \)
        }
      }
    }
}

// Demo 3: Color panels
DemoCard\(title = "Demo 3: Color Panels"\) {
    Box\(
      modifier = Modifier
        .fillMaxWidth\(\)
        .height\(120.dp\)
        .clip\(RoundedCornerShape\(16.dp\)\)
        .background\(MaterialTheme.colorScheme.secondaryContainer\),
      contentAlignment = Alignment.Center
    \) {
      Row\(
        horizontalArrangement = Arrangement.spacedBy\(12.dp\),
        verticalAlignment = Alignment.CenterVertically
      \) {
        Box\(
          modifier = Modifier
            .size\(60.dp\)
            .clip\(RoundedCornerShape\(12.dp\)\)
            .background\(MaterialTheme.colorScheme.primary\)
        \)
        Box\(
          modifier = Modifier
            .size\(60.dp\)
            .clip\(RoundedCornerShape\(12.dp\)\)
            .background\(MaterialTheme.colorScheme.tertiary\)
        \)
        Box\(
          modifier = Modifier
            .size\(60.dp\)
            .clip\(RoundedCornerShape\(12.dp\)\)
            .background\(MaterialTheme.colorScheme.error\)
        \)
      }
    }
}
```

```
            // 信息卡片
            Spacer\(modifier = Modifier.height\(8.dp\)\)
            Card\(
                modifier = Modifier.fillMaxWidth\(\),
                colors = CardDefaults.cardColors\(
                    containerColor = MaterialTheme.colorScheme.surfaceVariant
                \)
            \) {
                Column\(modifier = Modifier.padding\(16.dp\)\) {
                    Text\(
                        text = "Library Info",
                        style = MaterialTheme.typography.titleSmall,
                        fontWeight = FontWeight.SemiBold
                    \)
                    Spacer\(modifier = Modifier.height\(4.dp\)\)
                    Text\(
                        text = "• Library: kyant/backdrop \(io.github.kyant0\)\\n" +
                                "• Version: 1.0.0-rc02\\n" +
                                "• Compose BOM: 2024.09.00 \(Compose 1.7.x\)\\n" +
                                "• Min SDK: 30 \(Android 11+\)\\n" +
                                "• Package: com.kyant.backdrop",
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    \)
                }
            }

            Spacer\(modifier = Modifier.height\(32.dp\)\)
        }
    }
  }
}

@Composable
private fun DemoCard\(
    title: String,
    content: @Composable \(\) -> Unit
\) {
    Card\(
        modifier = Modifier.fillMaxWidth\(\),
        shape = RoundedCornerShape\(16.dp\),
        colors = CardDefaults.cardColors\(
            containerColor = MaterialTheme.colorScheme.surface
        \),
        elevation = CardDefaults.cardElevation\(defaultElevation = 2.dp\)
    \) {
        Column\(modifier = Modifier.padding\(16.dp\)\) {
            Text\(
                text = title,
                style = MaterialTheme.typography.titleMedium,
                fontWeight = FontWeight.SemiBold,
                modifier = Modifier.padding\(bottom = 12.dp\)
            \)
            content\(\)
        }
```

```
    }
}
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/ui/PredictiveBack.kt =====
package com.lightningstudio.watchrss.phone.ui

import androidx.activity.compose.PredictiveBackHandler
import androidx.compose.animation.core.animate
import androidx.compose.animation.core.tween
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.BoxScope
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableFloatStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.rememberUpdatedState
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.graphicsLayer
import androidx.compose.ui.unit.dp
import kotlinx.coroutines.CancellationException

const val PREDICTIVE_BACK_EXIT_PROGRESS = 2f
const val PREDICTIVE_BACK_EXIT_ANIMATION_MS = 140
const val PREDICTIVE_BACK_CANCEL_ANIMATION_MS = 480

@Composable
fun PredictiveBackSurface\(
    onBack: \(\) -> Unit,
    modifier: Modifier = Modifier,
    enabled: Boolean = true,
    onBeforeBack: suspend \(\) -> Unit = {},
    content: @Composable BoxScope.\(\) -> Unit
\) {
    var backProgress by remember { mutableFloatStateOf\(0f\) }
    val onBeforeBackState = rememberUpdatedState\(onBeforeBack\)
    val onBackState = rememberUpdatedState\(onBack\)

    PredictiveBackHandler\(enabled = enabled\) { backEvents ->
        try {
            backEvents.collect { backEvent ->
                backProgress = backEvent.progress.coerceIn\(0f, 1f\)
            }
            animate\(
                initialValue = backProgress,
                targetValue = PREDICTIVE_BACK_EXIT_PROGRESS,
                animationSpec = tween\(PREDICTIVE_BACK_EXIT_ANIMATION_MS\)
            \) { value, _ ->
                backProgress = value
            }
            onBeforeBackState.value\(\)
            onBackState.value\(\)
        } catch \(exception: CancellationException\) {
```

```
            animate\(
                initialValue = backProgress,
                targetValue = 0f,
                animationSpec = tween\(PREDICTIVE_BACK_CANCEL_ANIMATION_MS\)
            \) { value, _ ->
                backProgress = value
            }
        }
    }

    Surface\(
        modifier = modifier
            .fillMaxSize\(\)
            .predictiveBackExitPreview\(backProgress\),
        color = MaterialTheme.colorScheme.background
    \) {
        Box\(modifier = Modifier.fillMaxSize\(\)\) {
            content\(\)
        }
    }
}

fun Modifier.predictiveBackExitPreview\(progress: Float\): Modifier {
    val previewProgress = progress.coerceIn\(0f, 1f\)
    val exitProgress = \(progress - 1f\).coerceIn\(0f, 1f\)
    if \(previewProgress <= 0f && exitProgress <= 0f\) return this
    return graphicsLayer {
        translationX = 96.dp.toPx\(\) * previewProgress + size.width * exitProgress
        scaleX = 1f - 0.04f * previewProgress
        scaleY = 1f - 0.04f * previewProgress
        alpha = \(1f - 0.16f * previewProgress\) * \(1f - exitProgress\)
    }
}
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/ui/theme/AppCard.kt =====
package com.lightningstudio.watchrss.phone.ui.theme

import android.graphics.BlurMaskFilter
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.BoxScope
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.LocalContentColor
import androidx.compose.material3.MaterialTheme
import androidx.compose.runtime.Composable
import androidx.compose.runtime.CompositionLocalProvider
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.draw.drawBehind
import androidx.compose.ui.draw.shadow
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.Paint
```

```
import androidx.compose.ui.graphics.PaintingStyle
import androidx.compose.ui.graphics.Shape
import androidx.compose.ui.graphics.drawscope.drawIntoCanvas
import androidx.compose.ui.graphics.lerp
import androidx.compose.ui.graphics.nativeCanvas
import androidx.compose.ui.unit.Dp
import androidx.compose.ui.unit.dp
import androidx.compose.foundation.isSystemInDarkTheme

/**
 * 应用统一的卡片设计系统:
 * - 渐变光影质感
 * - 阴影悬浮感
 * - 边缘轻微高光立体感
 */

private val CardCornerRadius = 16.dp
private val CardElevation = 8.dp

@Composable
fun appCardGradient\(\): Brush {
    val isDark = isSystemInDarkTheme\(\)
    return if \(isDark\) {
        Brush.verticalGradient\(
            colors = listOf\(
                DarkCardStart,
                DarkCardEnd
            \)
        \)
    } else {
        Brush.verticalGradient\(
            colors = listOf\(
                LightCardStart,
                LightCardEnd
            \)
        \)
    }
}

@Composable
fun appCardHighlightBorder\(\): Color {
    return if \(isSystemInDarkTheme\(\)\) CardHighlightDark else CardHighlightLight
}

@Composable
fun appPinnedListCardGradient\(\): Brush {
    val isDark = isSystemInDarkTheme\(\)
    return if \(isDark\) {
        Brush.verticalGradient\(
            colors = listOf\(
                lerp\(DarkCardStart, Color.White, 0.08f\),
                lerp\(DarkCardEnd, Color.White, 0.06f\)
            \)
        \)
    } else {
```

```
        Brush.verticalGradient\(
            colors = listOf\(
                lerp\(LightCardStart, Color.Black, 0.06f\),
                lerp\(LightCardEnd, Color.Black, 0.04f\)
            \)
        \)
    }
}

/**
 * 统一的应用卡片组件
 */
@Composable
fun AppCard\(
    modifier: Modifier = Modifier,
    shape: Shape = RoundedCornerShape\(CardCornerRadius\),
    elevation: Dp = CardElevation,
    interactionModifier: Modifier = Modifier,
    content: @Composable BoxScope.\(\) -> Unit
\) {
    val highlightColor = appCardHighlightBorder\(\)

    Box\(
        modifier = modifier
            .shadow\(
                elevation = elevation,
                shape = shape,
                spotColor = PrimaryRed.copy\(alpha = 0.15f\),
                ambientColor = PrimaryRed.copy\(alpha = 0.05f\)
            \)
            .clip\(shape\)
            .background\(appCardGradient\(\)\)
            .border\(
                width = 0.5.dp,
                color = highlightColor,
                shape = shape
            \)
            .then\(interactionModifier\)
    \) {
        CompositionLocalProvider\(
            LocalContentColor provides MaterialTheme.colorScheme.onSurface
        \) {
            content\(\)
        }
    }
}

/**
 * 主色强调卡片\(用于重要操作或状态\)
 */
@Composable
fun AppPrimaryCard\(
    modifier: Modifier = Modifier,
    shape: Shape = RoundedCornerShape\(CardCornerRadius\),
    interactionModifier: Modifier = Modifier,
```

```
    content: @Composable BoxScope.\(\) -> Unit
\) {
    val isDark = isSystemInDarkTheme\(\)
    val gradient = if \(isDark\) {
        Brush.verticalGradient\(
            colors = listOf\(
                DarkPrimary.copy\(alpha = 0.20f\),
                DarkPrimaryContainer.copy\(alpha = 0.10f\)
            \)
        \)
    } else {
        Brush.verticalGradient\(
            colors = listOf\(
                PrimaryRed.copy\(alpha = 0.12f\),
                GradientEnd.copy\(alpha = 0.08f\)
            \)
        \)
    }

    Box\(
        modifier = modifier
            .shadow\(
                elevation = 12.dp,
                shape = shape,
                spotColor = if \(isDark\) DarkPrimary.copy\(alpha = 0.30f\) else PrimaryRed.copy\(alpha = 0.25f\),
                ambientColor = if \(isDark\) DarkPrimary.copy\(alpha = 0.10f\) else PrimaryRed.copy\(alpha = 0.1f\)
            \)
            .clip\(shape\)
            .background\(gradient\)
            .border\(
                width = 0.5.dp,
                color = if \(isDark\) DarkPrimary.copy\(alpha = 0.5f\) else PrimaryRed.copy\(alpha = 0.2f\),
                shape = shape
            \)
            .then\(interactionModifier\)
    \) {
        CompositionLocalProvider\(
            LocalContentColor provides MaterialTheme.colorScheme.onSurface
        \) {
            content\(\)
        }
    }
}

/**
 * 列表项卡片\(更紧凑\) —— 含方向性提亮的液态玻璃轮廓
 */
@Composable
fun AppListCard\(
    modifier: Modifier = Modifier,
    interactionModifier: Modifier = Modifier,
    backgroundBrush: Brush? = null,
    content: @Composable BoxScope.\(\) -> Unit
\) {
    val isDark = isSystemInDarkTheme\(\)
```

```
val highlightAlpha = if \(isDark\) 0.15f else 0.50f
val glowAlpha = if \(isDark\) 0.10f else 0.40f
val shape = RoundedCornerShape\(12.dp\)
val cardBackground = backgroundBrush ?: appCardGradient\(\)

Box\(
   modifier = modifier
      .shadow\(
         elevation = 4.dp,
         shape = shape,
         spotColor = PrimaryRed.copy\(alpha = 0.15f\),
         ambientColor = PrimaryRed.copy\(alpha = 0.05f\)
      \)
      .clip\(shape\)
      .background\(cardBackground\)
      .drawBehind {
         val width = size.width
         val height = size.height
         val r = 12.dp.toPx\(\)

         // 第 1 层：外发光描边（BlurMaskFilter 柔化轮廓）
         val glowPaint = Paint\(\).apply {
            color = Color.White.copy\(alpha = glowAlpha\)
            style = PaintingStyle.Stroke
            strokeWidth = 2.dp.toPx\(\)
            asFrameworkPaint\(\).maskFilter = BlurMaskFilter\(
               6.dp.toPx\(\), BlurMaskFilter.Blur.NORMAL
            \)
         }
         drawIntoCanvas { canvas ->
            canvas.nativeCanvas.drawRoundRect\(
               0f, 0f, width, height, r, r,
               glowPaint.asFrameworkPaint\(\)
            \)
         }

         // 第 2 层：菲涅尔式方向性高光 —— 45° 光源从左上照射
         // 只提亮顶部和左侧边缘，背面自然暗淡
         val highlightPaint = Paint\(\).apply {
            color = Color.White.copy\(alpha = highlightAlpha\)
            style = PaintingStyle.Stroke
            strokeWidth = 1.2.dp.toPx\(\)
            asFrameworkPaint\(\).maskFilter = BlurMaskFilter\(
               2.dp.toPx\(\), BlurMaskFilter.Blur.NORMAL
            \)
         }
         drawIntoCanvas { canvas ->
            canvas.nativeCanvas.drawLine\(
               r, 0f, width - r, 0f,
               highlightPaint.asFrameworkPaint\(\)
            \)
            canvas.nativeCanvas.drawLine\(
               0f, r, 0f, height - r,
               highlightPaint.asFrameworkPaint\(\)
            \)
```

```
            }
        }
        .then\(interactionModifier\)
    \) {
        CompositionLocalProvider\(
            LocalContentColor provides MaterialTheme.colorScheme.onSurface
        \) {
            content\(\)
        }
    }
}

/**
 * 兼容 Material3 Card 的包装
 */
@Composable
fun AppMaterialCard\(
    modifier: Modifier = Modifier,
    content: @Composable \(\) -> Unit
\) {
    val isDark = isSystemInDarkTheme\(\)
    Card\(
        modifier = modifier,
        shape = RoundedCornerShape\(CardCornerRadius\),
        colors = CardDefaults.cardColors\(
            containerColor = if \(isDark\) DarkSurfaceElevated else LightSurface
        \),
        elevation = CardDefaults.cardElevation\(
            defaultElevation = CardElevation
        \)
    \) {
        Box\(
            modifier = Modifier.background\(appCardGradient\(\)\)
        \) {
            content\(\)
        }
    }
}
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/ui/theme/Color.kt =====
package com.lightningstudio.watchrss.phone.ui.theme

import androidx.compose.ui.graphics.Color

val BrandPrimary = Color\(0xFF006C5B\)
val BrandOnPrimary = Color\(0xFFFFFFFF\)
val BrandPrimaryContainer = Color\(0xFF82F8D6\)
val BrandOnPrimaryContainer = Color\(0xFF002019\)

val BrandSecondary = Color\(0xFF4A635D\)
val BrandOnSecondary = Color\(0xFFFFFFFF\)
val BrandSecondaryContainer = Color\(0xFFCDE8DF\)
val BrandOnSecondaryContainer = Color\(0xFF06201A\)

val BrandTertiary = Color\(0xFF456179\)
val BrandOnTertiary = Color\(0xFFFFFFFF\)
```

```
val BrandTertiaryContainer = Color\(0xFFCBE6FF\)
val BrandOnTertiaryContainer = Color\(0xFF001E31\)

val LightBackground = Color\(0xFFFAFDF9\)
val LightSurface = Color\(0xFFFAFDF9\)
val LightSurfaceContainer = Color\(0xFFEFF4EF\)
val LightSurfaceVariant = Color\(0xFFDCE5DE\)
val LightOutline = Color\(0xFF707973\)
val OnBackgroundLight = Color\(0xFF191C1A\)
val OnSurfaceVariantLight = Color\(0xFF404943\)

val DarkPrimary = Color\(0xFF64DBBA\)
val DarkOnPrimary = Color\(0xFF00382D\)
val DarkPrimaryContainer = Color\(0xFF005143\)
val DarkOnPrimaryContainer = Color\(0xFF82F8D6\)

val DarkSecondary = Color\(0xFFB1CCC3\)
val DarkOnSecondary = Color\(0xFF1C3530\)
val DarkSecondaryContainer = Color\(0xFF334B46\)
val DarkOnSecondaryContainer = Color\(0xFFCDE8DF\)

val DarkTertiary = Color\(0xFFADCAE6\)
val DarkOnTertiary = Color\(0xFF133348\)
val DarkTertiaryContainer = Color\(0xFF2C4961\)
val DarkOnTertiaryContainer = Color\(0xFFCBE6FF\)

val DarkBackground = Color\(0xFF101412\)
val DarkSurface = Color\(0xFF101412\)
val DarkSurfaceElevated = Color\(0xFF1C211E\)
val DarkSurfaceVariant = Color\(0xFF404943\)
val DarkOutline = Color\(0xFF89938C\)
val OnBackgroundDark = Color\(0xFFE0E4DF\)
val OnSurfaceVariantDark = Color\(0xFFC0C9C2\)

val Error = Color\(0xFFBA1A1A\)
val OnError = Color\(0xFFFFFFFF\)
val ErrorContainer = Color\(0xFFFFDAD6\)
val OnErrorContainer = Color\(0xFF410002\)
val DarkError = Color\(0xFFFFB4AB\)
val DarkOnError = Color\(0xFF690005\)
val DarkErrorContainer = Color\(0xFF93000A\)
val DarkOnErrorContainer = Color\(0xFFFFDAD6\)

val LightCardStart = LightSurfaceContainer
val LightCardEnd = LightSurface
val DarkCardStart = DarkSurfaceElevated
val DarkCardEnd = DarkSurface
val CardHighlightLight = Color\(0x99FFFFFF\)
val CardHighlightDark = Color\(0x334DD6B8\)

// Legacy aliases kept for older screens/components that still import these names.
val PrimaryRed = BrandPrimary
val PrimaryRedLight = BrandSecondary
val PrimaryRedDark = BrandTertiary
val GradientStart = BrandPrimary
```

```
val GradientMid = BrandPrimaryContainer
val GradientEnd = BrandSecondaryContainer
val OnPrimary = BrandOnPrimary
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/ui/theme/RoundedInteractions.kt =====
package com.lightningstudio.watchrss.phone.ui.theme

import androidx.compose.foundation.ExperimentalFoundationApi
import androidx.compose.foundation.LocalIndication
import androidx.compose.foundation.clickable
import androidx.compose.foundation.combinedClickable
import androidx.compose.foundation.interaction.MutableInteractionSource
import androidx.compose.foundation.interaction.PressInteraction
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.runtime.Composable
import androidx.compose.runtime.DisposableEffect
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.mutableStateListOf
import androidx.compose.runtime.remember
import androidx.compose.ui.Modifier
import androidx.compose.ui.composed
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Shape
import androidx.compose.ui.semantics.Role
import androidx.compose.ui.unit.dp
import androidx.lifecycle.Lifecycle
import androidx.lifecycle.LifecycleEventObserver
import androidx.lifecycle.compose.LocalLifecycleOwner

fun Modifier.roundedClickable\(
    shape: Shape = RoundedCornerShape\(12.dp\),
    enabled: Boolean = true,
    onClickLabel: String? = null,
    role: Role? = null,
    onClick: \(\) -> Unit
\): Modifier = composed {
    val interactionSource = rememberResettingInteractionSource\(\)
    clip\(shape\).clickable\(
        interactionSource = interactionSource,
        indication = LocalIndication.current,
        enabled = enabled,
        onClickLabel = onClickLabel,
        role = role,
        onClick = onClick
    \)
}

@OptIn\(ExperimentalFoundationApi::class\)
fun Modifier.roundedCombinedClickable\(
    shape: Shape = RoundedCornerShape\(12.dp\),
    enabled: Boolean = true,
    onClickLabel: String? = null,
    role: Role? = null,
    onLongClickLabel: String? = null,
    onLongClick: \(\(\) -> Unit\)? = null,
    onDoubleClick: \(\(\) -> Unit\)? = null,
```

```
    hapticFeedbackEnabled: Boolean = true,
    onClick: \(\) -> Unit
\): Modifier = composed {
    val interactionSource = rememberResettingInteractionSource\(\)
    clip\(shape\).combinedClickable\(
        interactionSource = interactionSource,
        indication = LocalIndication.current,
        enabled = enabled,
        onClickLabel = onClickLabel,
        role = role,
        onLongClickLabel = onLongClickLabel,
        onLongClick = onLongClick,
        onDoubleClick = onDoubleClick,
        hapticFeedbackEnabled = hapticFeedbackEnabled,
        onClick = onClick
    \)
}

@Composable
private fun rememberResettingInteractionSource\(\): MutableInteractionSource {
    val interactionSource = remember { MutableInteractionSource\(\) }
    val activePresses = remember { mutableStateListOf<PressInteraction.Press>\(\) }
    val lifecycleOwner = LocalLifecycleOwner.current

    fun cancelActivePresses\(\) {
        activePresses.toList\(\).forEach { press ->
            interactionSource.tryEmit\(PressInteraction.Cancel\(press\)\)
        }
        activePresses.clear\(\)
    }

    LaunchedEffect\(interactionSource\) {
        interactionSource.interactions.collect { interaction ->
            when \(interaction\) {
                is PressInteraction.Press -> activePresses.add\(interaction\)
                is PressInteraction.Release -> activePresses.remove\(interaction.press\)
                is PressInteraction.Cancel -> activePresses.remove\(interaction.press\)
            }
        }
    }

    DisposableEffect\(interactionSource, lifecycleOwner\) {
        val observer = LifecycleEventObserver { _, event ->
            if \(event == Lifecycle.Event.ON_PAUSE || event == Lifecycle.Event.ON_STOP\) {
                cancelActivePresses\(\)
            }
        }
        lifecycleOwner.lifecycle.addObserver\(observer\)
        onDispose {
            cancelActivePresses\(\)
            lifecycleOwner.lifecycle.removeObserver\(observer\)
        }
    }

    return interactionSource
```

```
}
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/ui/theme/Theme.kt =====
package com.lightningstudio.watchrss.phone.ui.theme

import android.os.Build
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.darkColorScheme
import androidx.compose.material3.dynamicDarkColorScheme
import androidx.compose.material3.dynamicLightColorScheme
import androidx.compose.material3.lightColorScheme
import androidx.compose.runtime.Composable
import androidx.compose.ui.platform.LocalContext

private val LightColors = lightColorScheme\(
    primary = BrandPrimary,
    onPrimary = BrandOnPrimary,
    primaryContainer = BrandPrimaryContainer,
    onPrimaryContainer = BrandOnPrimaryContainer,
    secondary = BrandSecondary,
    onSecondary = BrandOnSecondary,
    secondaryContainer = BrandSecondaryContainer,
    onSecondaryContainer = BrandOnSecondaryContainer,
    tertiary = BrandTertiary,
    onTertiary = BrandOnTertiary,
    tertiaryContainer = BrandTertiaryContainer,
    onTertiaryContainer = BrandOnTertiaryContainer,
    background = LightBackground,
    onBackground = OnBackgroundLight,
    surface = LightSurface,
    onSurface = OnBackgroundLight,
    surfaceVariant = LightSurfaceVariant,
    onSurfaceVariant = OnSurfaceVariantLight,
    outline = LightOutline,
    error = Error,
    onError = OnError,
    errorContainer = ErrorContainer,
    onErrorContainer = OnErrorContainer
\)

private val DarkColors = darkColorScheme\(
    primary = DarkPrimary,
    onPrimary = DarkOnPrimary,
    primaryContainer = DarkPrimaryContainer,
    onPrimaryContainer = DarkOnPrimaryContainer,
    secondary = DarkSecondary,
    onSecondary = DarkOnSecondary,
    secondaryContainer = DarkSecondaryContainer,
    onSecondaryContainer = DarkOnSecondaryContainer,
    tertiary = DarkTertiary,
    onTertiary = DarkOnTertiary,
    tertiaryContainer = DarkTertiaryContainer,
    onTertiaryContainer = DarkOnTertiaryContainer,
    background = DarkBackground,
    onBackground = OnBackgroundDark,
```

```
    surface = DarkSurface,
    onSurface = OnBackgroundDark,
    surfaceVariant = DarkSurfaceVariant,
    onSurfaceVariant = OnSurfaceVariantDark,
    outline = DarkOutline,
    error = DarkError,
    onError = DarkOnError,
    errorContainer = DarkErrorContainer,
    onErrorContainer = DarkOnErrorContainer
\)

@Composable
fun WatchRssPhoneTheme\(
    dynamicColor: Boolean = true,
    content: @Composable \(\) -> Unit
\) {
    val darkTheme = isSystemInDarkTheme\(\)
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if \(darkTheme\) dynamicDarkColorScheme\(context\) else dynamicLightColorScheme\(context\)
        }
        darkTheme -> DarkColors
        else -> LightColors
    }

    MaterialTheme\(
        colorScheme = colorScheme,
        content = content
    \)
}
// ===== File: app/src/main/java/com/lightningstudio/watchrss/phone/viewmodel/MainViewModelFactory.kt =====
package com.lightningstudio.watchrss.phone.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import com.lightningstudio.watchrss.phone.connection.bluetooth.PhoneBluetoothSyncManager
import com.lightningstudio.watchrss.phone.data.backup.WatchRssBackupService
import com.lightningstudio.watchrss.phone.data.repo.PhoneCompanionRepository
import com.lightningstudio.watchrss.phone.data.telemetry.PhoneUsageTelemetry

class MainViewModelFactory\(
    private val repository: PhoneCompanionRepository,
    private val bluetoothSyncManager: PhoneBluetoothSyncManager,
    private val usageTelemetry: PhoneUsageTelemetry,
    private val backupService: WatchRssBackupService
\) : ViewModelProvider.Factory {
    @Suppress\("UNCHECKED_CAST"\)
    override fun <T : ViewModel> create\(modelClass: Class<T>\): T {
        if \(modelClass.isAssignableFrom\(MainViewModel::class.java\)\) {
            return MainViewModel\(repository, bluetoothSyncManager, usageTelemetry, backupService\) as T
        }
        throw IllegalArgumentException\("Unknown ViewModel class: ${modelClass.name}"\)
    }
}
```